

TECHNICAL SPECIFICATIONS

Prachya Ph. (Puy)

Software Solution Consultant

เป้าหมาย

Hi-Level Design
Architecture

DB Schema
API Spec

Conceptual
Detailed Design

TECHNICAL SPEC

Functional & Non-functional Requirement



ระบบต้องทำอะไรได้บ้าง

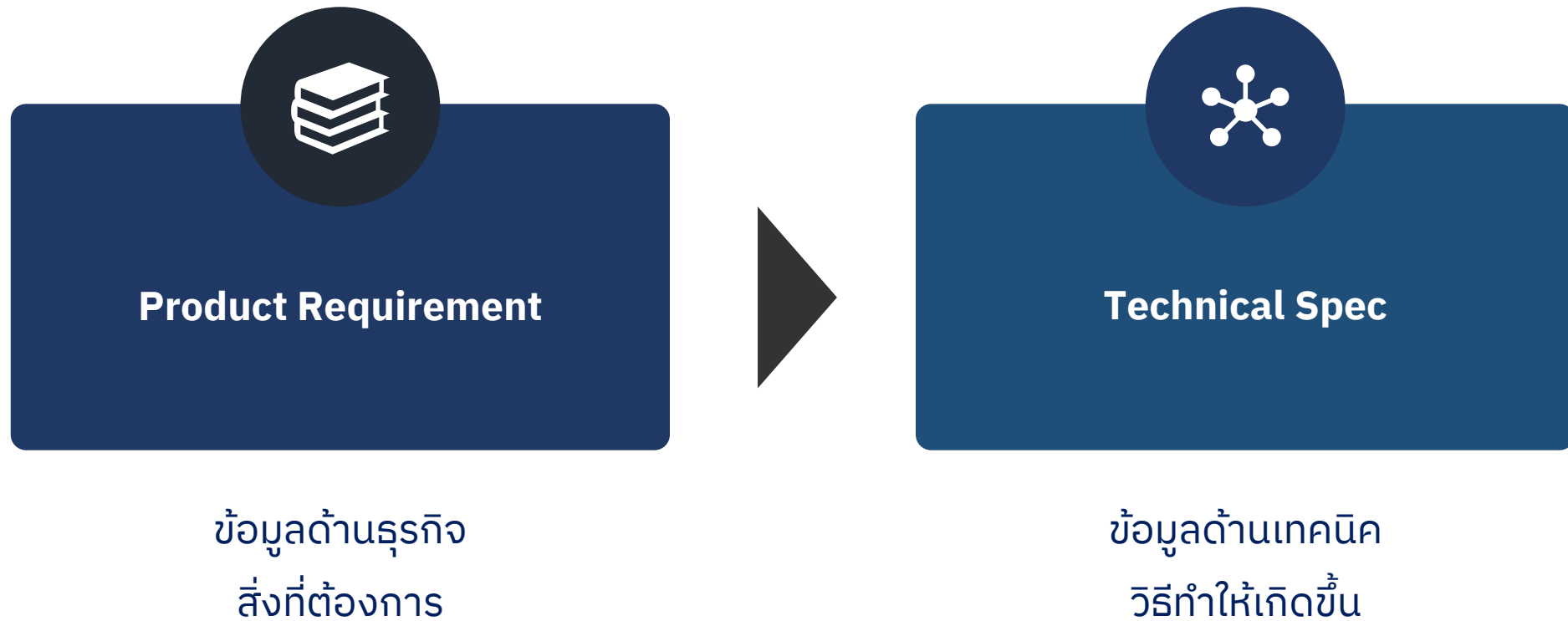
เช่น สั่งร้านอาหารเครื่องดื่มได้



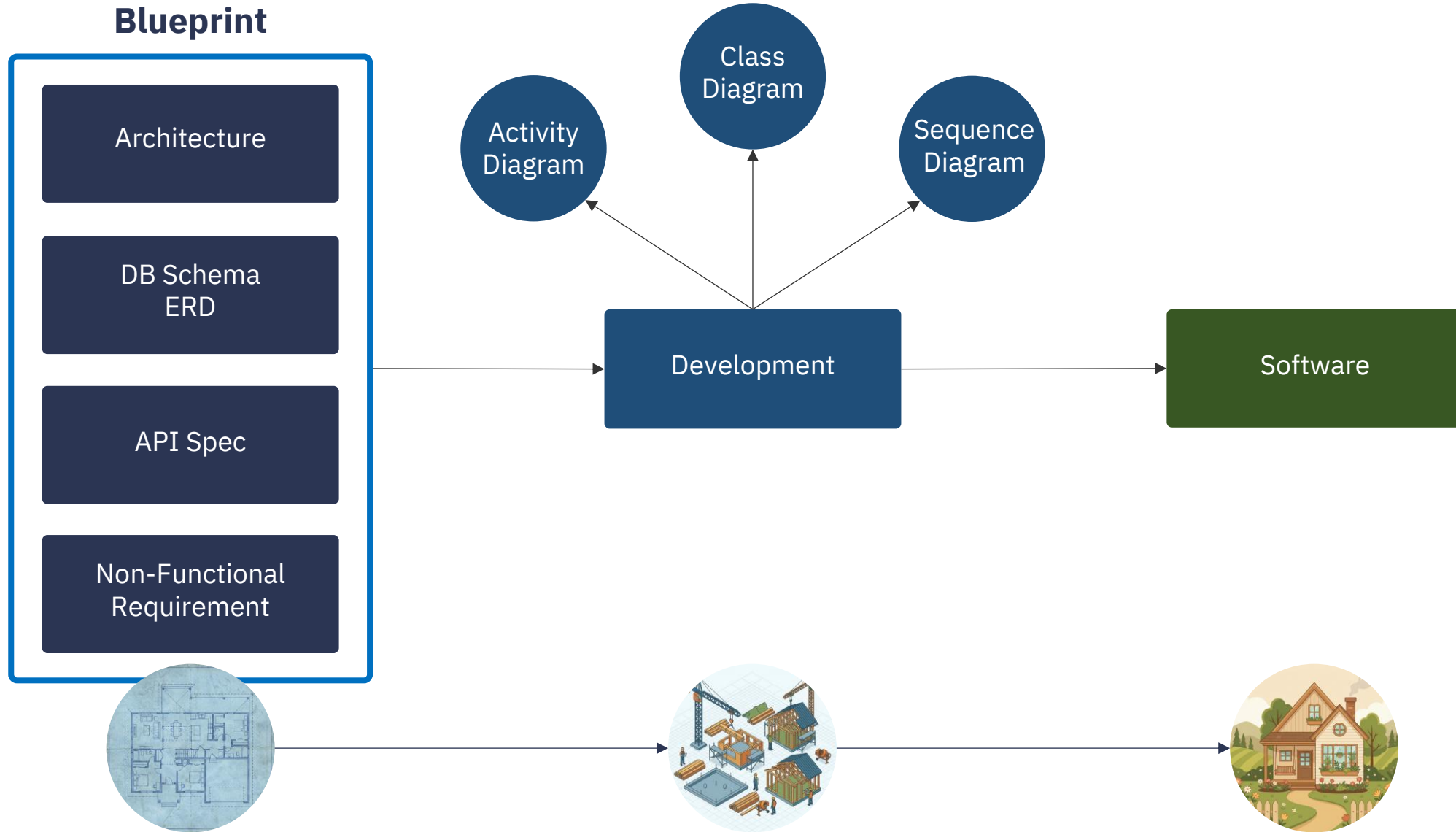
ระบบต้องทำงานดีแค่ไหน

เช่น เร็ว, ปลอดภัย,
รองรับคนใช้พร้อมกันได้เยอะ

ทำไมต้องแยก Technical Spec ออกจาก Product Requirement



Technical Specification Blueprint



ทำไมต้องมี Technical Spec



เพิ่มความชัดเจน
(Context)
ในการทำงานร่วมกับ AI



ควบคุม Architecture &
Coding Standards ให้
สม่ำเสมอ



ประหยัด Tokens
ลดการทำงานซ้ำซ้อน



แบ่งงานสั่งให้ AI
ทำที่ละชิ้นได้ง่ายขึ้น

ข้อมูลควรมีใน Tech Spec เบื้องต้น

High-Level Design

System Architecture

ภาพรวมโครงสร้างสถาปัตยกรรมระบบ

Data Flow

เส้นทางการเดินทางของข้อมูล
ของส่วนที่สำคัญ

Technology Stack

เทคโนโลยีที่ใช้

Authentication & Authorization

รูปแบบการยืนยันตัวตน (Authentication)
และ การกำหนดสิทธิ์การเข้าถึง (Authorization)

Non-Functional Requirement

ความต้องการที่ไม่ใช่
ฟังก์ชันการทำงานหลัก
ในมิติต่างๆ เช่น
Scalability,
Performance,
Security, และอื่นๆ

Detailed Design

Database Schema

โครงสร้างฐานข้อมูล

Business Logic & Validation Rules

ตรรกะและลำดับขั้นตอน
อธิบายการทำงาน

Edge Cases & Error Handling Spec

กรณีข้อผิดพลาด

API Spec

โครงสร้างการรับส่งข้อมูล
แต่ละ API

Security

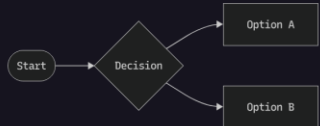
ความปลอดภัย

Mermaid.js (Diagram ที่ใช้งานบ่อย)

เครื่องมือช่วยวาด Diagram ซึ่งสามารถเปลี่ยน Text ใน Markdown ให้กลายเป็น Diagram ได้

Flowchart

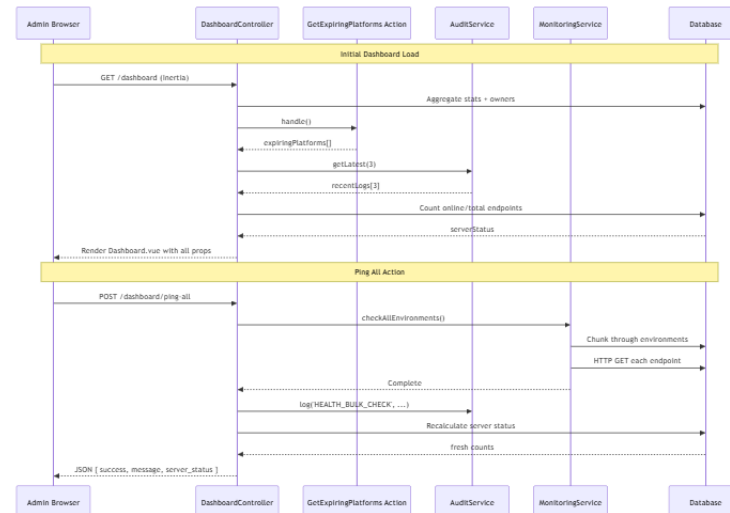
```
1 flowchart LR
2   A(["Start"]) --> B{"Decision"}
3   B --> C["Option A"]
4   B --> D["Option B"]
```



Sequence Diagram

```
sequenceDiagram
    participant Admin as Admin Browser
    participant Dash as DashboardController
    participant GetExpiring as GetExpiringPlatforms Action
    participant AuditSvc as AuditService
    participant MonitorSvc as MonitoringService
    participant DB as Database

    Note over Admin,DB: Initial Dashboard Load
    Admin->>Dash: GET /dashboard (Inertia)
    Dash->>GetExpiring: handle()
    GetExpiring->>AuditSvc: expiringPlatforms()
    AuditSvc->>DB: getLabels()
    DB-->>AuditSvc: recentsLog()
    AuditSvc-->>MonitorSvc: Count online/total endpoints
    MonitorSvc-->>DB: serverStatus
    DB-->>MonitorSvc: Render Dashboard.vue with all props
    MonitorSvc-->>Admin: Ping All Action
    Admin->>Dash: POST /dashboard/ping-all
    Dash->>GetExpiring: checkAllEnvironments()
    GetExpiring->>AuditSvc: Chunk through environments
    AuditSvc->>DB: HTTP GET each endpoint
    DB-->>AuditSvc: Complete
    AuditSvc-->>MonitorSvc: log(HEALTH_CHECK, ...)
    MonitorSvc-->>DB: Recalculate server status
    DB-->>MonitorSvc: fresh counts
    MonitorSvc-->>Admin: JSON [ success, message, server_status ]
```



ERDiagram

```
erDiagram
    OWNERS ||--o{ PLATFORMS : "owns"
    OWNERS ||--o{ DEVICES : "owns fleet"
    CREDENTIALS ||--o{ PLATFORMS : "assigned to (Global)"
    USERS ||--o{ AUDIT_LOGS : "performs"
    PLATFORMS ||--o{ PLATFORM_ENVIRONMENTS : "has environments"
    PLATFORM_ENVIRONMENTS ||--o{ ENVIRONMENT_VARIABLES : "stores vars"
    PLATFORM_ENVIRONMENTS ||--o{ DEPLOYMENT_LOGS : "tracks deploys"
    PLATFORMS ||--o{ ACTIVATION_LOGS : "monitors"
    DEVICES ||--o{ ACTIVATION_LOGS : "generates"

    PLATFORMS {
        string id PK
        string name
        string platform_code
        string contact_email
        string deleted_at
    }

    PLATFORM_ENVIRONMENTS {
        string id PK
        string platform_id FK
        string name
        string type
        string status
        string deleted_at
    }

    ENVIRONMENT_VARIABLES {
        string id PK
        string platform_environment_id FK
        string key
        string value
        string deleted_at
    }

    DEPLOYMENT_LOGS {
        string id PK
        string platform_environment_id FK
        string type
        string status
        string deleted_at
    }

    ACTIVATION_LOGS {
        string id PK
        string platform_id FK
        string device_id FK
        string status
        string deleted_at
    }
```

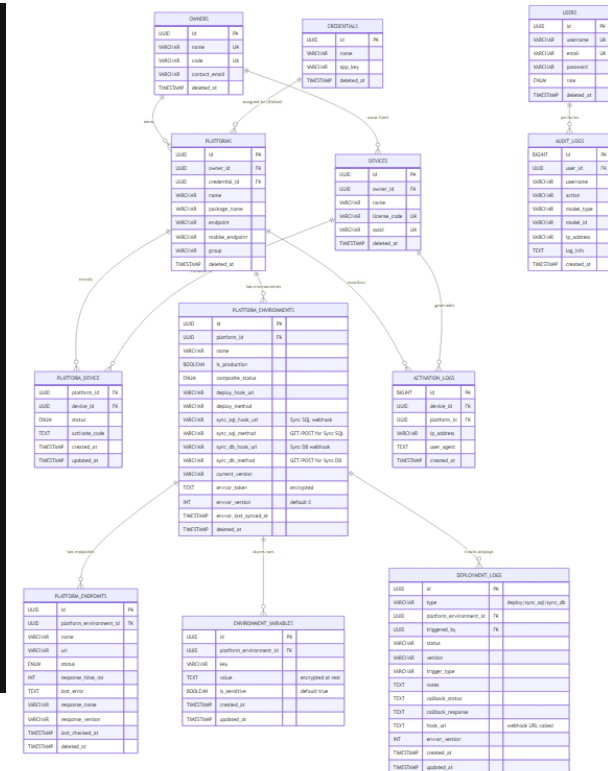
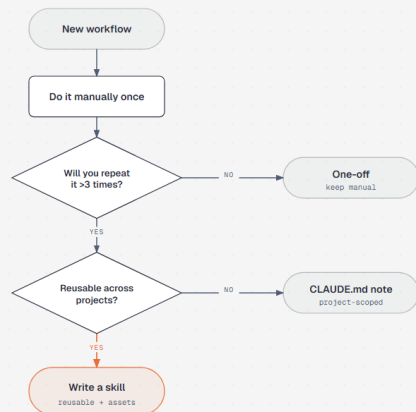


Diagram Design (ทำให้ Diagram อ่านง่ายขึ้น)

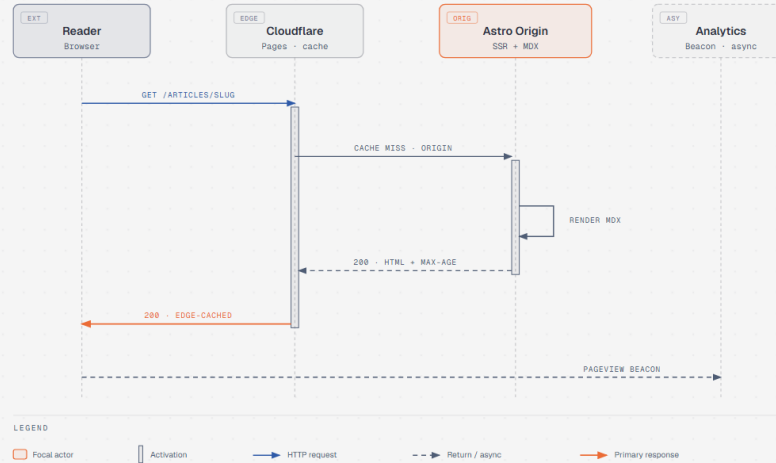
เครื่องมือช่วยแปลง mermaid diagram ให้ดูดีขึ้น

แนะนำให้ใช้ตอนต้องการจะรีวิว หรือต้องการนำ diagram ไปใช้งานต่อที่อื่น

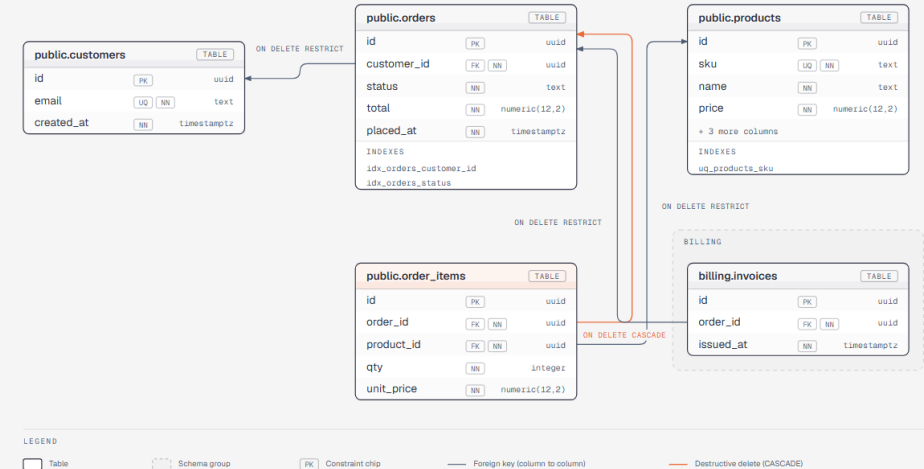
Flowchart



Sequence Diagram

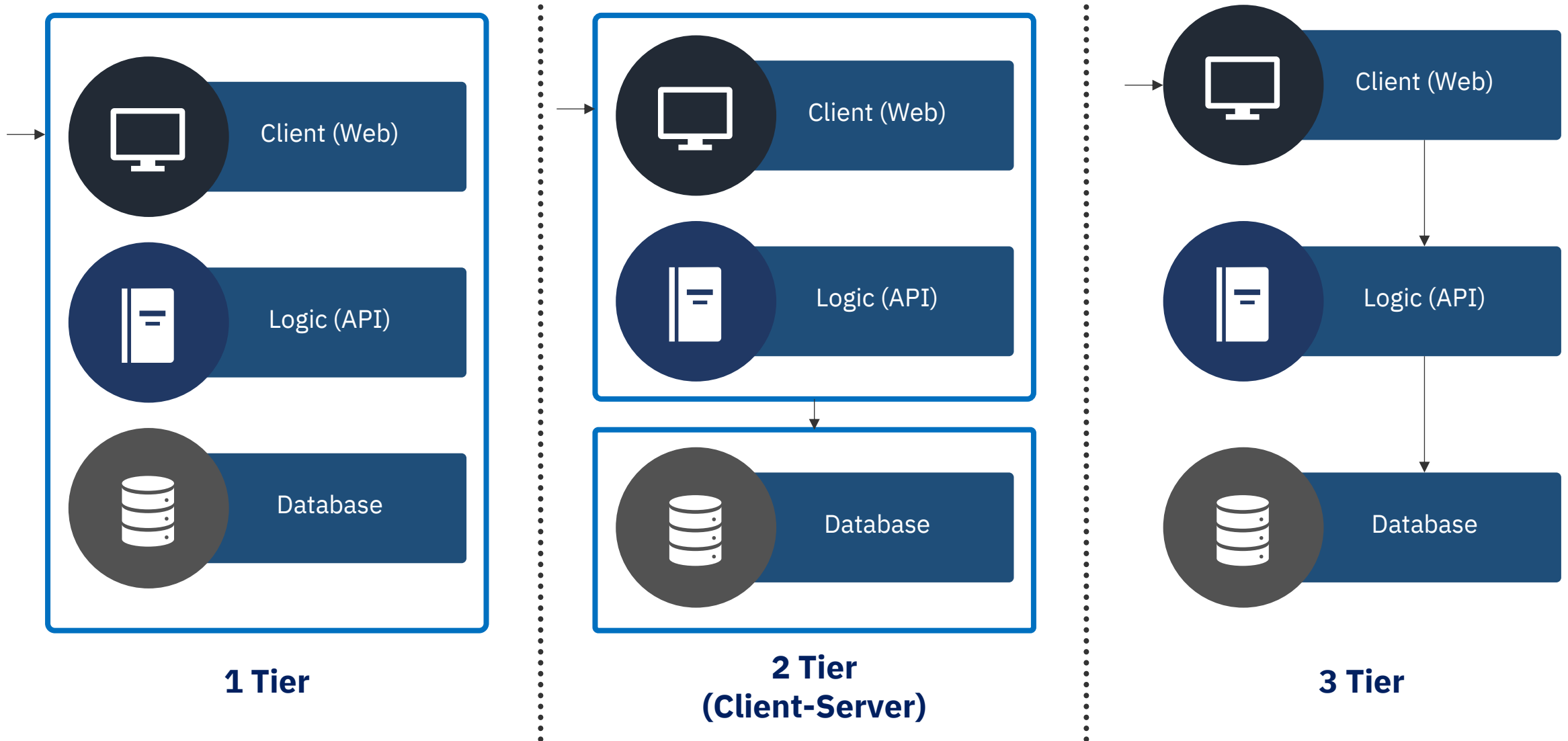


ERDiagram

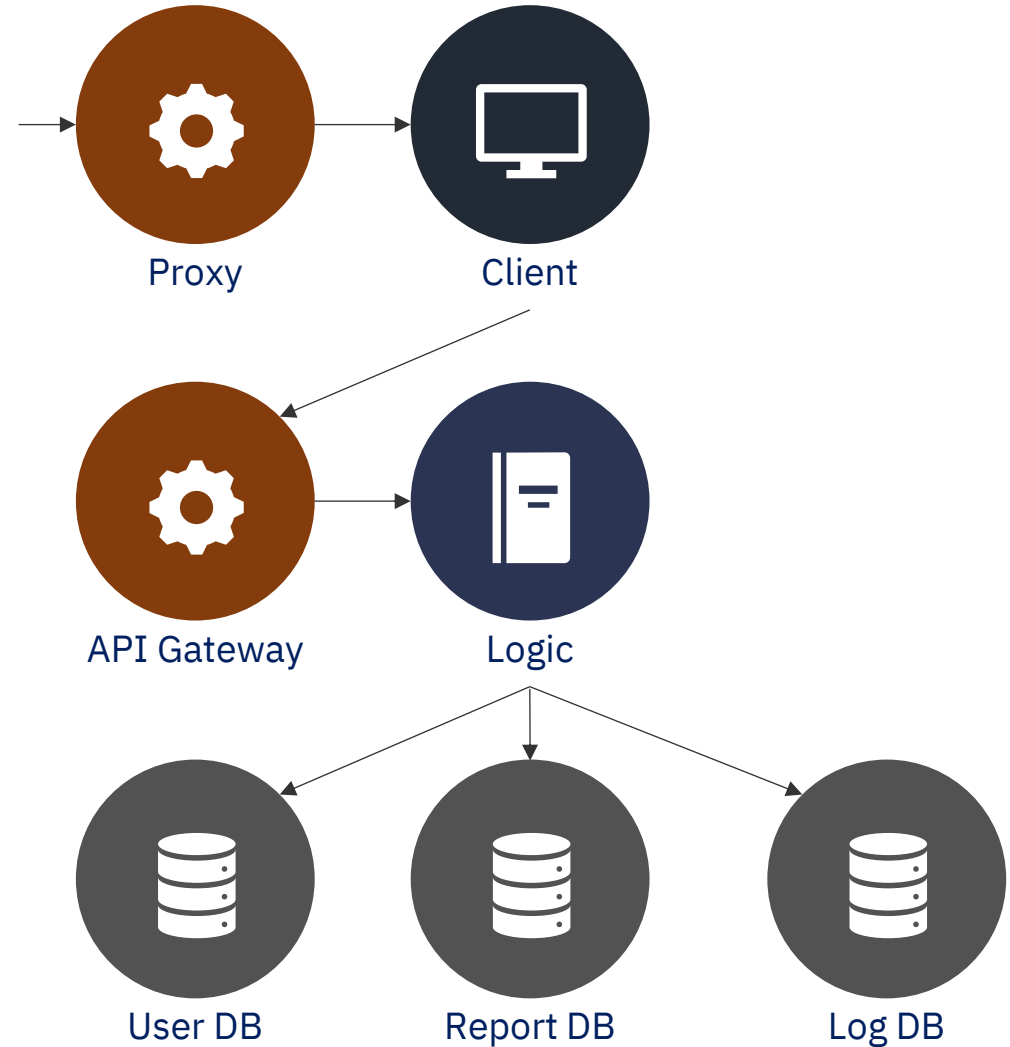
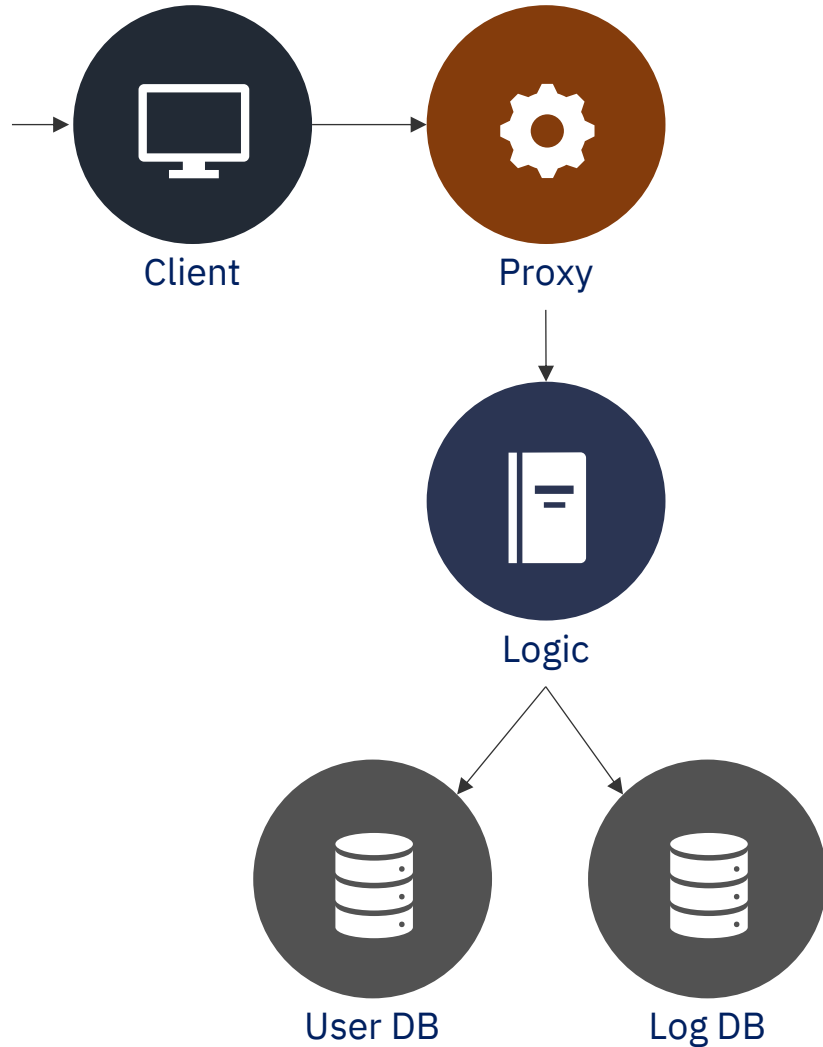


HIGH-LEVEL ARCHITECTURE

System Architecture

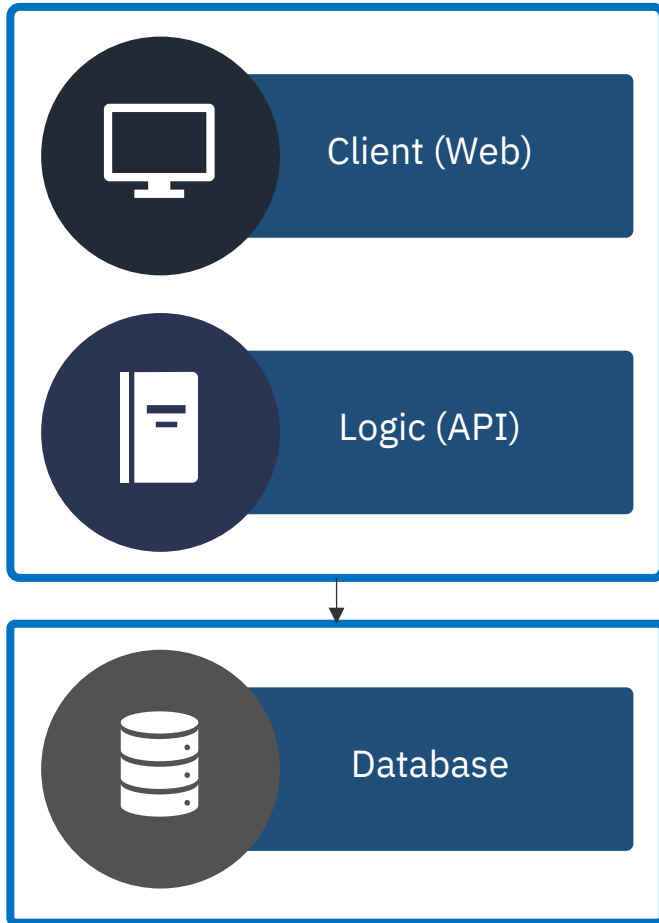


System Architecture

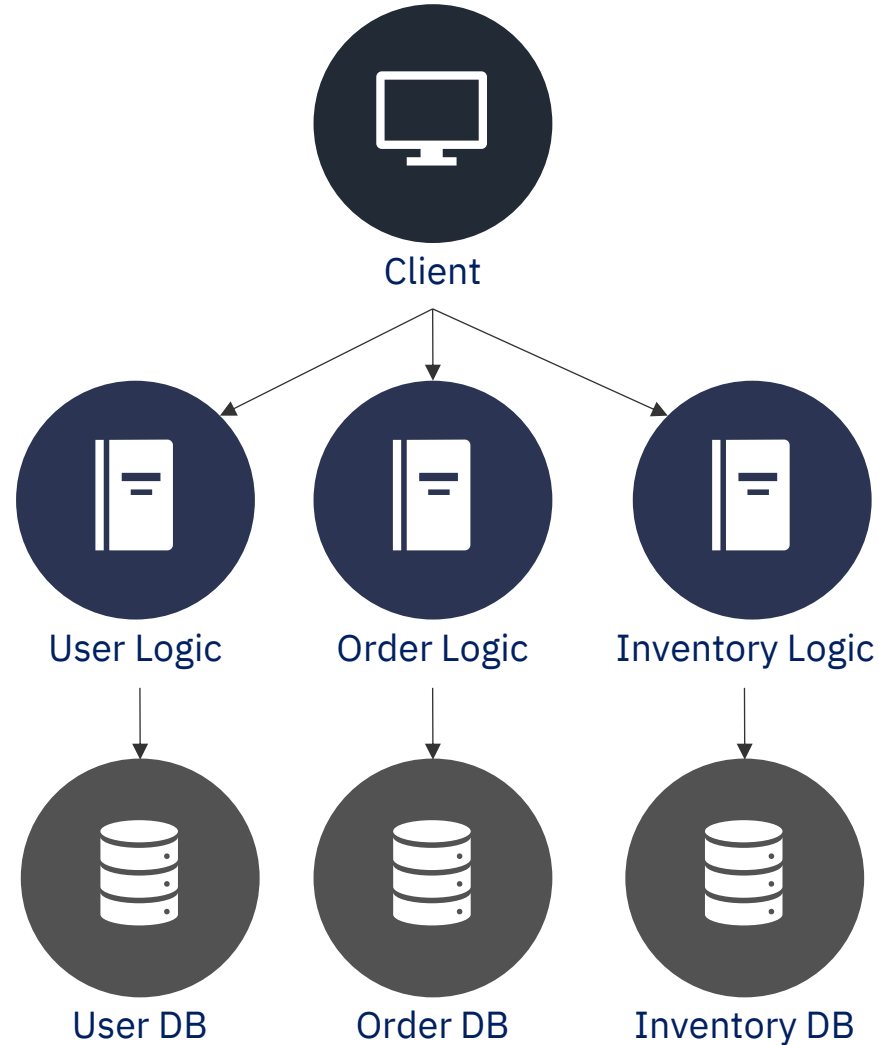


N Tier

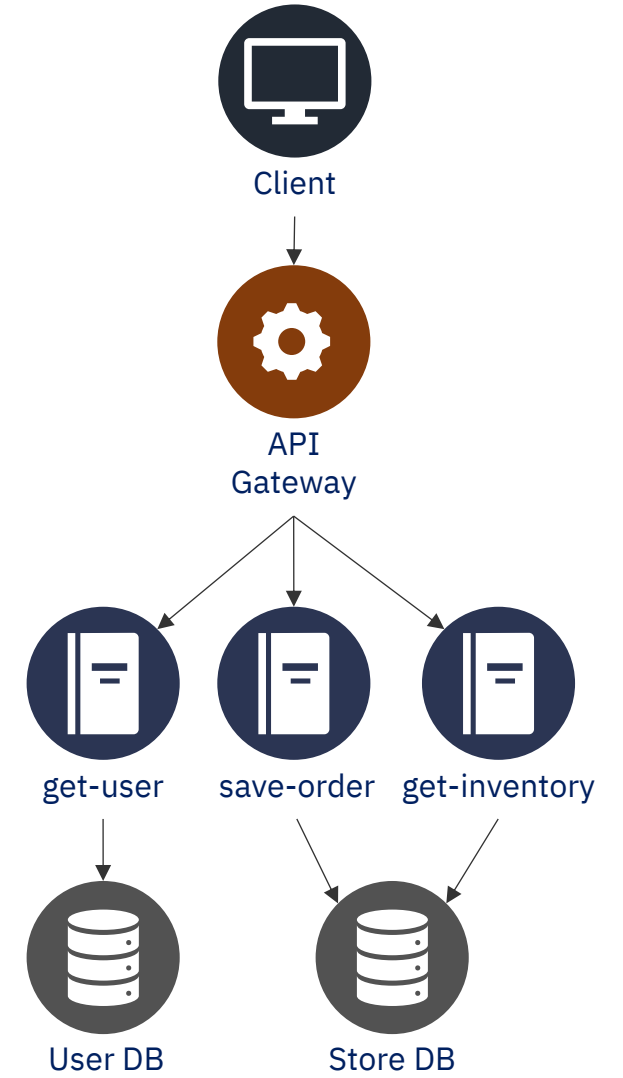
System Architecture



Monolithic



Microservice



Serverless

Technology Stack



Frontend / Client

- ส่วนหน้าบ้านที่ผู้ใช้มองเห็น
- เทคโนโลยีที่รันบนอุปกรณ์ของผู้ใช้ (Web Browser / Mobile App)
- Web: HTML, CSS, JavaScript, TypeScript
- Frontend Framework: React, Next, Angular, Nuxt, Vue, Laravel
- Mobile Framework: Swift (iOS), Kotlin (Android), Flutter, React Native



Backend / Logic

- ส่วนระบบหลังบ้านและตรรกะการทำงาน
- เชื่อมต่อฐานข้อมูล
- Language & Framework: NodeJS (Express, NestJS), Python (Django/FastAPI), Java (Spring Boot, Spring MVC), Go, C# (.NET), PHP (Laravel)



Database / Storage

- ส่วนจัดเก็บข้อมูลของระบบ
- Relational Database (SQL): PostgreSQL, MySQL, MariaDB
- NoSQL Database: MongoDB, Redis (Caching), Firestore

Technology Stack Example

MERN Stack

สำหรับ Web App

- M: Mongo DB
- E: Express.js
- R: React
- N: Node.js

Modern Full-Stack

สำหรับ Web App

- Next.js (TypeScript)
- Tailwind CSS
- Prisma ORM
- PostgreSQL
- Vercel

Enterprise Grade

สำหรับระบบขนาดใหญ่

- Java (Spring Boot),
C# (.NET Core),
Go
- Angular / React
- PostgreSQL / Oracle
- Kubernetes

Data Flow

"ข้อมูลเคลื่อนที่อย่างไรในระบบ" ตั้งแต่จุดเริ่มต้นที่ข้อมูลถูกสร้างขึ้น ผ่านขั้นตอนการประมวลผล (Processing), การแปลงรูป (Transformation), การตรวจสอบ (Validation), การจัดเก็บลงฐานข้อมูล ไปจนถึงการส่งผลลัพธ์กลับไปยังผู้ใช้หรือระบบภายนอก



ลดความคลุมเครือ

แสดงให้เห็นชัดเจนว่า Service ไหน
คุยกับ Service ไหน ผ่านโปรโตคอล
อะไร (HTTP, WebSocket, gRPC,
Message Queue)



ระบุจุดคอขวดและจุดผิดพลาด (Bottlenecks & Failure Points)

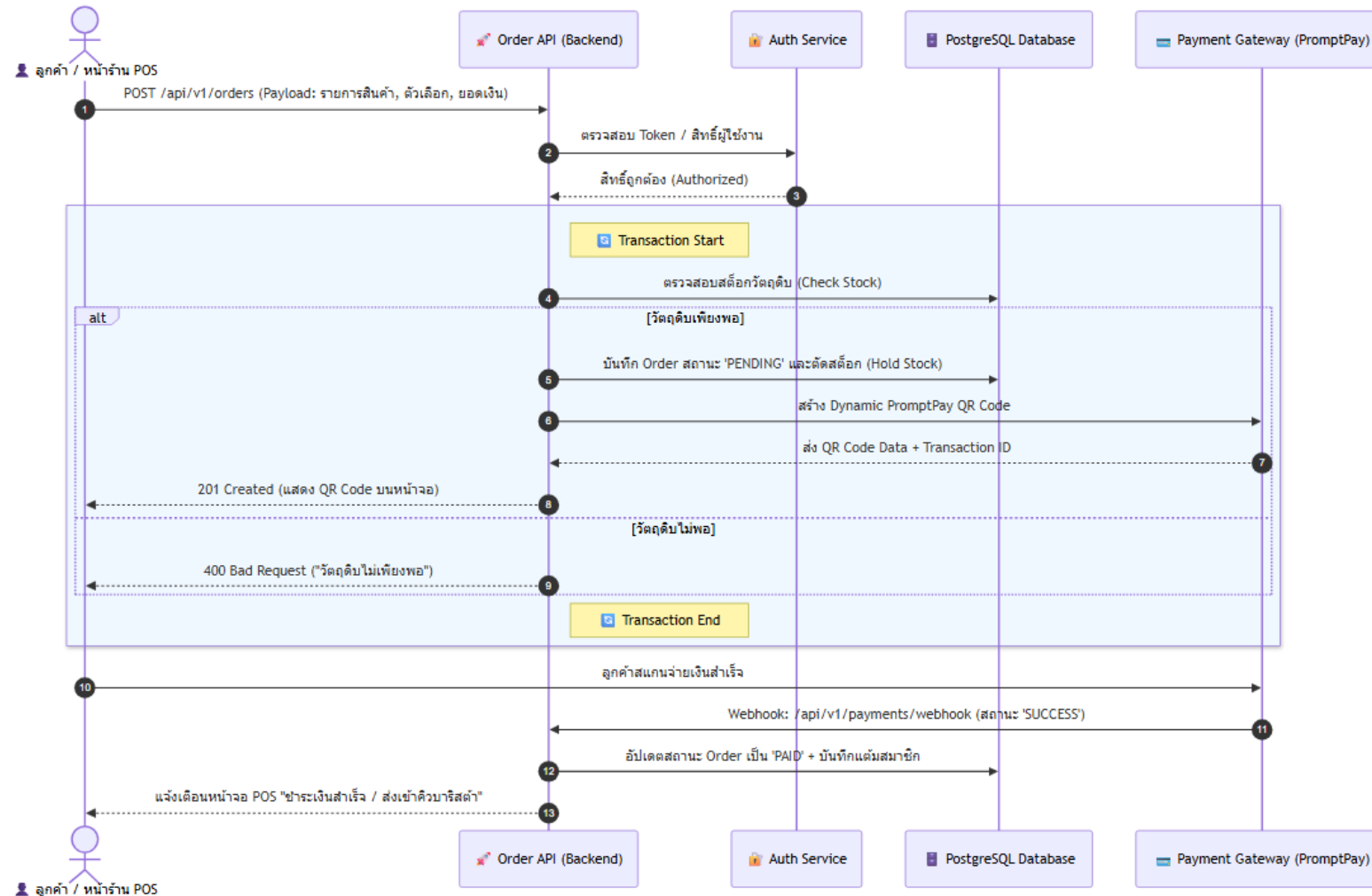
ช่วยให้ทีมเห็นว่าถ้าเกิดข้อผิดพลาด
ตรงจุดไหน ระบบต้องจัดการ
ย้อนกลับ (Rollback) หรือแจ้งเตือน
อย่างไร



เป็นพิมพ์เขียวให้ AI Agent

ช่วยให้ AI Agent เข้าใจขั้นตอนทาง
ตรรกะ (Step-by-Step Logic) และไม่
เขียนโค้ดลำดับขั้นตอนสำคัญ เช่น ลืมตัด
สต็อก หรือลืมบันทึก Log

Data Flow Example



Data Flow Sequence Diagram

Authentication & Authorization

การออกแบบระบบสำหรับ การยืนยันตัวตน และการกำหนดสิทธิ์ เพื่อ อธิบายนโยบาย และขอบเขตความปลอดภัย ของระบบ



Authentication การยืนยันตัวตน

กระบวนการตรวจสอบผู้ใช้งานว่าเป็นใคร
เพื่อเป็นการพิสูจน์ว่าเป็นตัวจริง ไม่ใช่คนอื่นแอบอ้าง
สำหรับเข้าใช้งาน



Authorization (Role & Permission) การอนุญาต / การให้สิทธิ์การเข้าถึง

กระบวนการตรวจสอบผู้ใช้งานว่าคนนั้นมีสิทธิ์เข้าถึงข้อมูล
หรือทำอะไรได้บ้าง เพื่อเป็นการอนุญาตให้เข้าถึงพื้นที่หรือ
คำสั่งเฉพาะ

รูปแบบของ Authentication (A1)

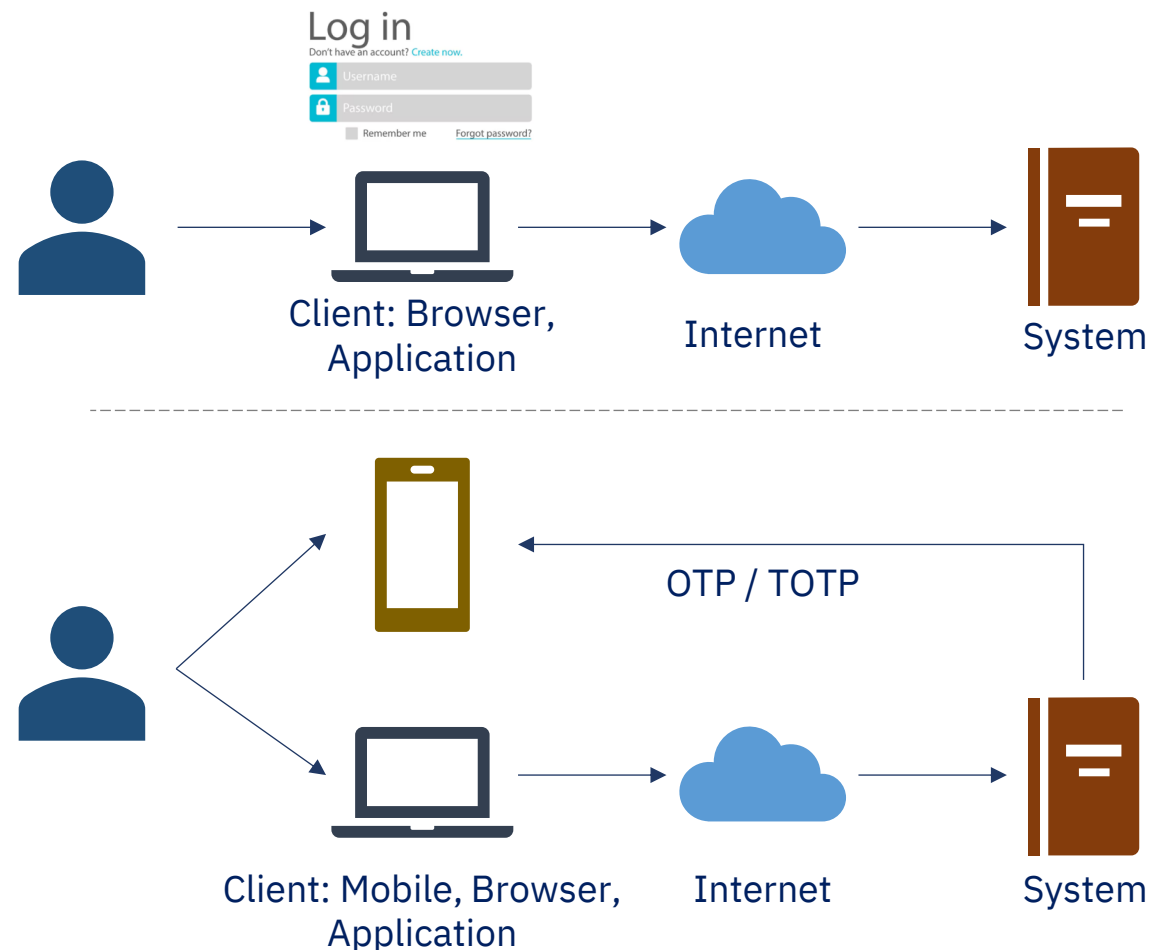
จำแนกตามปัจจัยการยืนยันตัวตน (Authentication Factors)

Knowledge-Based * (Something you know)

สิ่งที่ผู้ใช้รู้เพียงคนเดียว
เช่น รหัสผ่าน (Password), รหัส PIN,
คำถามก้นลิม

Possession-Based (Something you have)

SMS OTP, แอปสร้างรหัสชั่วคราว (TOTP เช่น
Google Authenticator), กุญแจฮาร์ดแวร์, บัตร
สมาร์ทการ์ด

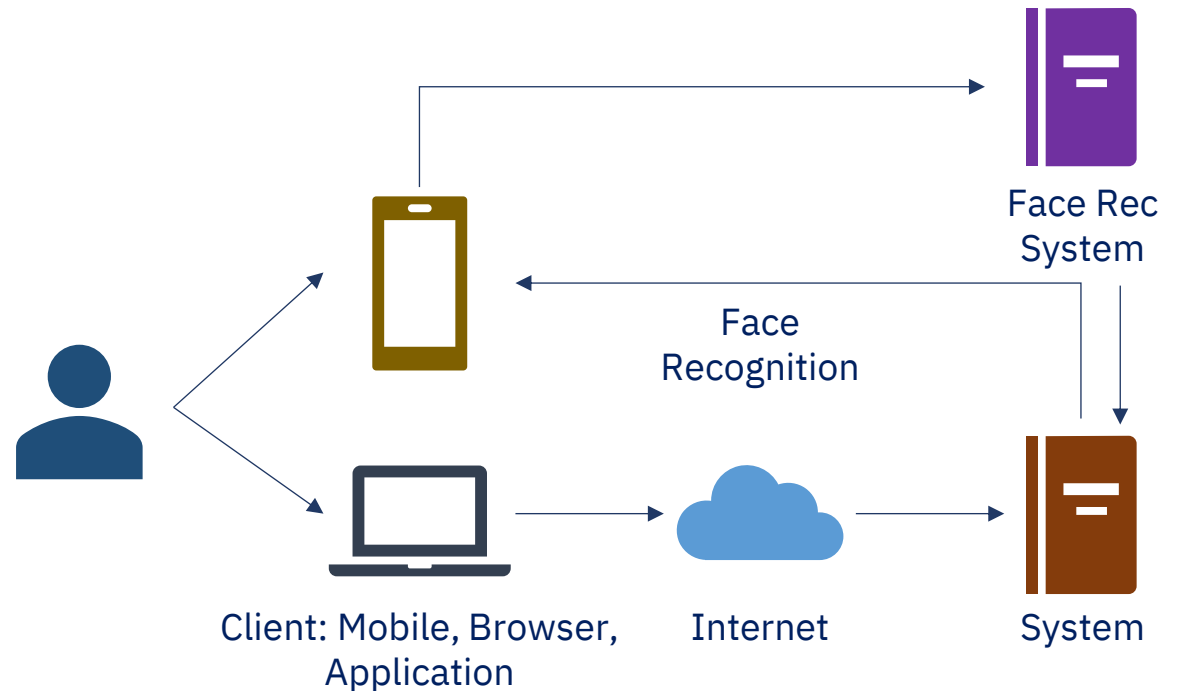


รูปแบบของ Authentication (A2)

จำแนกตามปัจจัยการยืนยันตัวตน
(Authentication Factors)

Inherence-Based (Something you are)

ข้อมูลทางชีวมิติ (Biometrics)
เช่น สแกนลายนิ้วมือ, สแกนใบหน้า
(Face ID), สแกนม่านตา

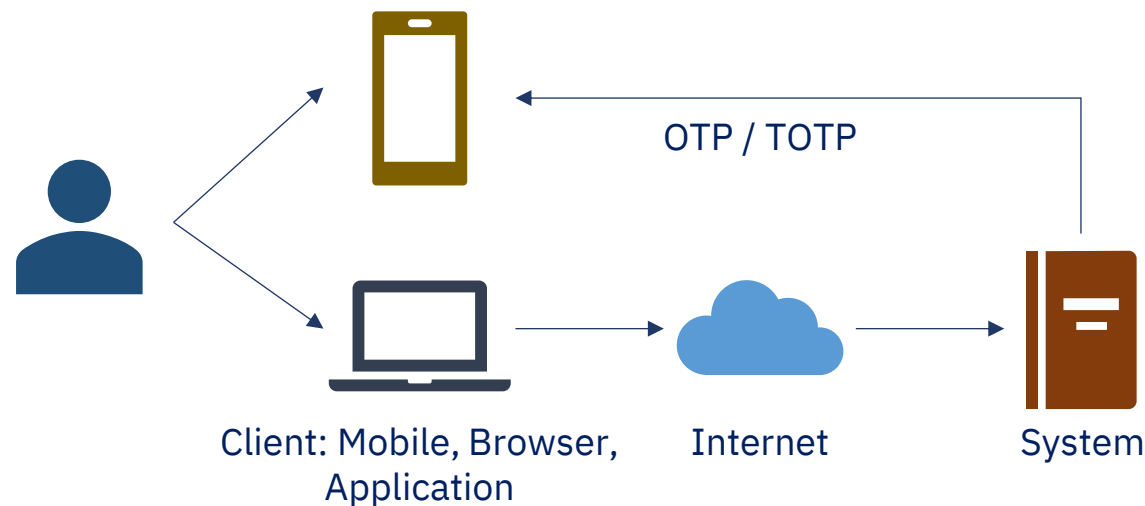


รูปแบบของ Authentication (A3)

จำแนกตามปัจจัยการยืนยันตัวตน
(Authentication Factors)

Multi-Factor Authentication (MFA / 2FA)

การผสมผสานตั้งแต่ 2 ปัจจัยขึ้นไป
(เช่น กรอกรหัสผ่าน + สแกนลายนิ้วมือ + ยืนยัน
OTP) เพื่อเพิ่มความปลอดภัย



Log in

Don't have an account? [Create now.](#)

Username

Password

☐ Remember me

[Forgot password?](#)

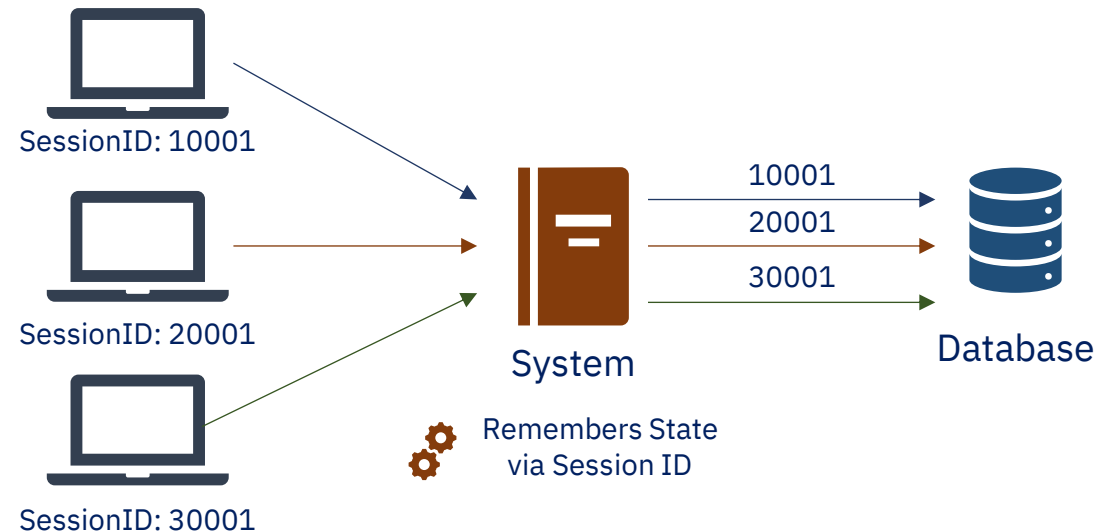
รูปแบบของ Authentication (B1)

จำแนกตามสถาปัตยกรรม และ กลไกการส่งข้อมูล
(Architectural Forms)

Session-Based (Stateful) *

เมื่อผู้ใช้ล็อกอินสำเร็จ เซิร์ฟเวอร์จะสร้าง Session เก็บไว้ในหน่วยความจำ/ฐานข้อมูล และส่ง Session ID กลับไปเก็บใน Cookie ของ Browser

ควบคุมการตัด Session ได้ทันทีจากฝั่งเซิร์ฟเวอร์
แต่รองรับการขยายระบบ (Scale-out) ได้ยากกว่า



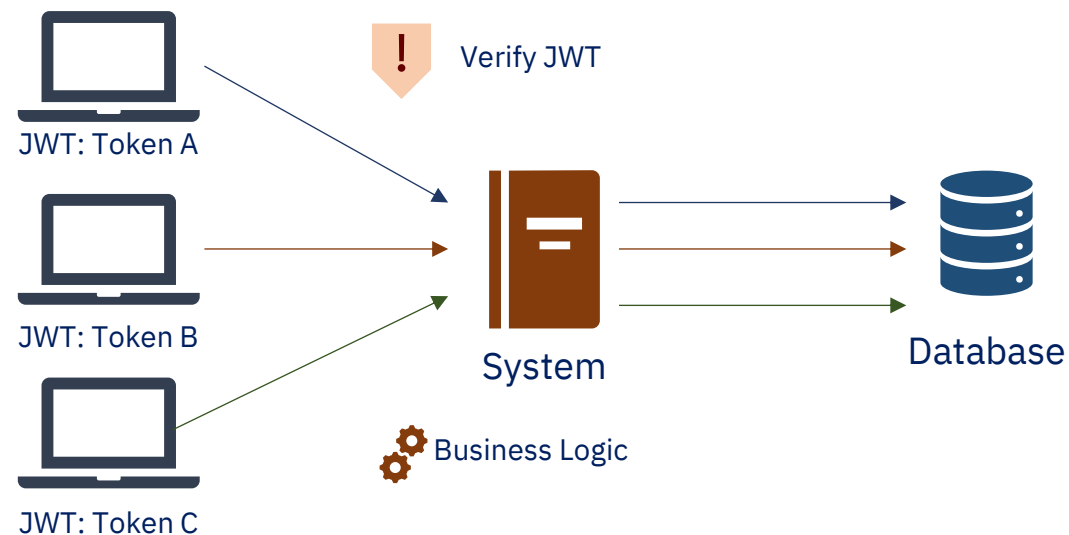
รูปแบบของ Authentication (B2)

จำแนกตามสถาปัตยกรรม และ กลไกการส่งข้อมูล
(Architectural Forms)

Token-Based / JWT (Stateless) *

เซิร์ฟเวอร์ออก JSON Web Token (JWT) ที่เข้ารหัสและมีลายเซ็นดิจิทัล (Digital Signature) ให้ Client นำไปแนบใน Request Header (Authorization: Bearer <token>)

เซิร์ฟเวอร์ไม่ต้องเก็บ State เหมาะกับ Microservices และ Mobile Apps แต่การส่ง Token นั้นทำได้ซับซ้อนกว่า



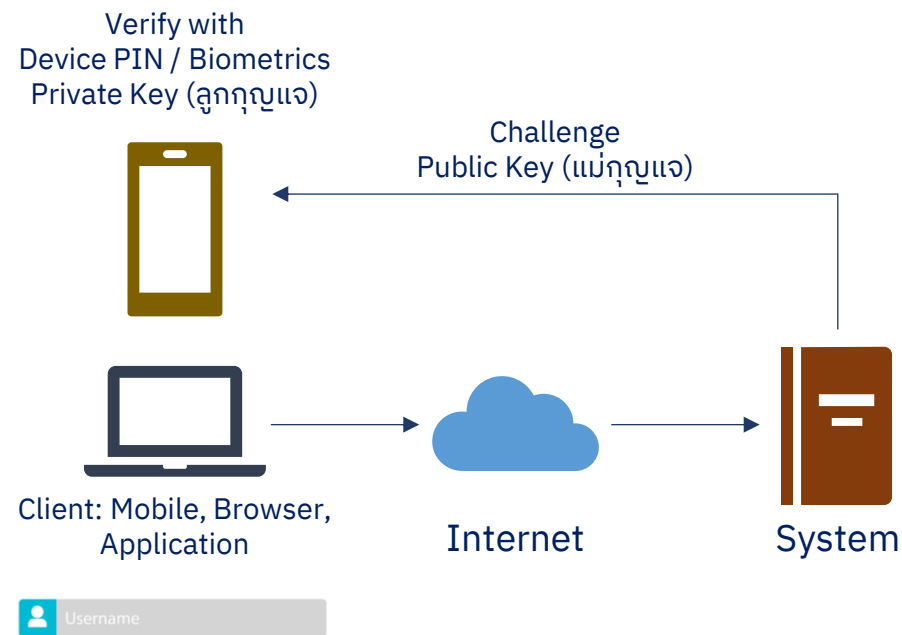
รูปแบบของ Authentication (B3)

จำแนกตามสถาปัตยกรรม และ กลไกการส่งข้อมูล
(Architectural Forms)

Passwordless & Passkeys (FIDO2 / WebAuthn)

ใช้การเข้ารหัสแบบกุญแจคู่ (Public/Private Key)
โดย Private Key ถูกเก็บอย่างปลอดภัยในชิปของอุปกรณ์
(เช่น iPhone, Android, Laptop) และปลดล็อกด้วย Biometrics

ป้องกันการถูกหลอกเอาข้อมูล (Phishing-resistant)
ได้สมบูรณ์แบบที่สุด

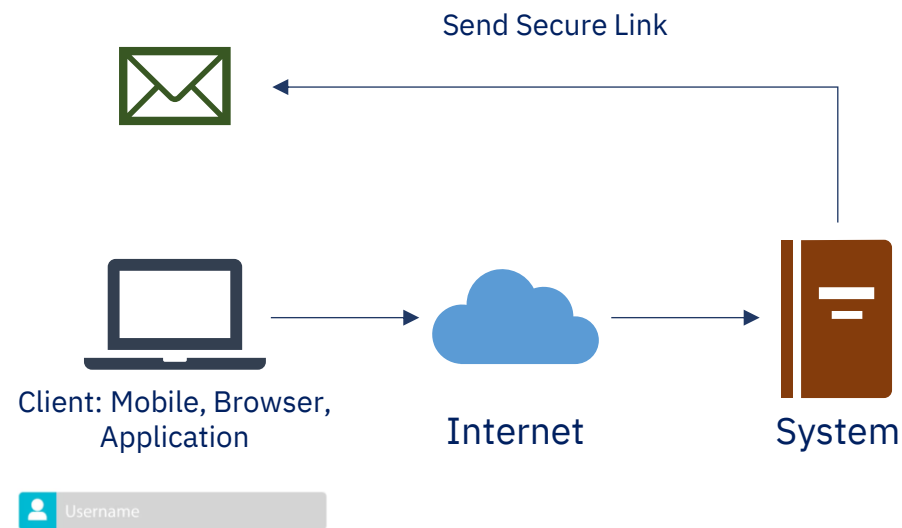


รูปแบบของ Authentication (B4)

จำแนกตามสถาปัตยกรรม และ กลไกการส่งข้อมูล
(Architectural Forms)

Magic Link

ส่งลิงก์ชั่วคราวที่มี Token ฝังอยู่ไปยังอีเมลหรือ SMS เมื่อกดลิงก์จะถือว่ายืนยันตัวตนสำเร็จทันที



รูปแบบของ Authentication (C)

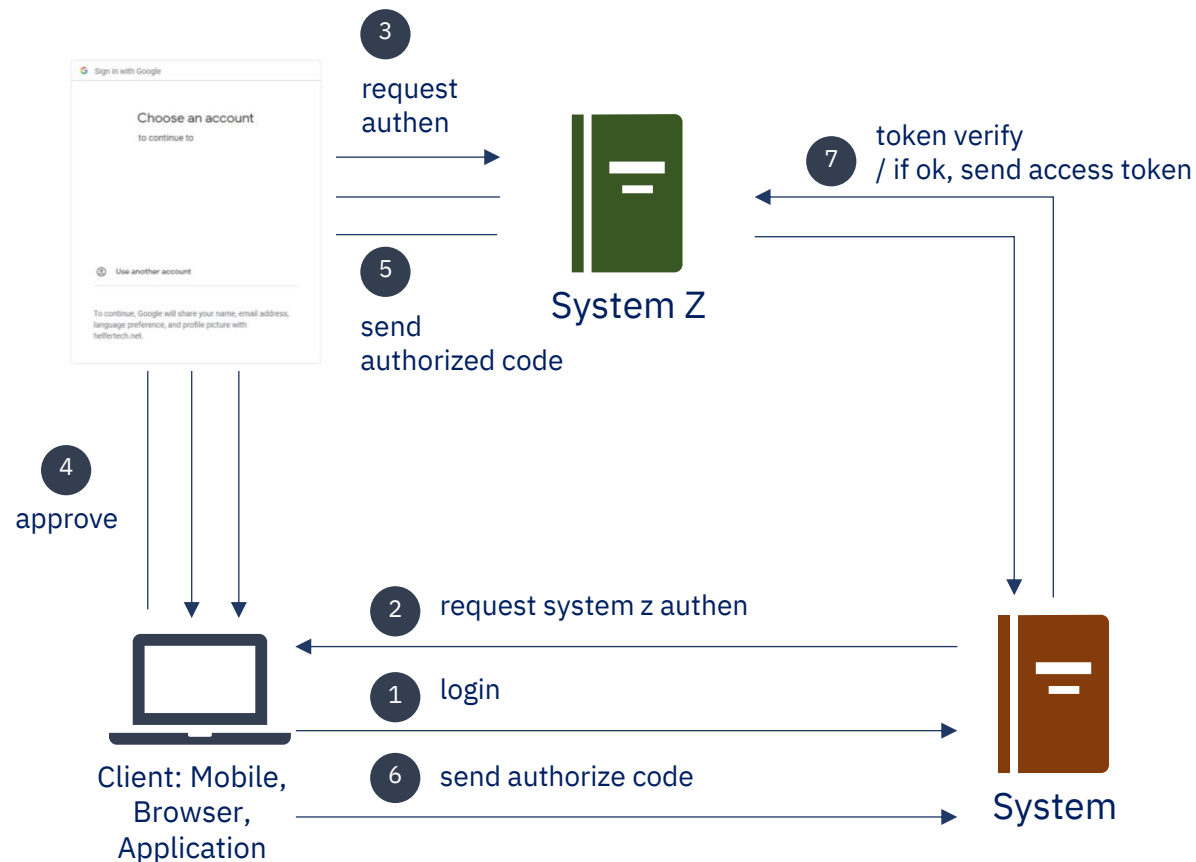
จำแนกตามการเชื่อมต่อข้ามระบบ (Federation & SSO)

Social Login & OpenID Connect (OIDC)

การยืนยันตัวตนผ่านผู้ให้บริการบุคคลที่สาม (Identity Provider) เช่น Log in with Google, Apple, LINE, หรือ Facebook

Single Sign-On (Enterprise SSO - SAML / OIDC)

ล็อกอินครั้งเดียวที่ส่วนกลาง (เช่น Okta, Azure AD / Entra ID, Keycloak) แล้วเข้าใช้งานระบบภายในองค์กรได้ทั้งหมดโดยไม่ต้องล็อกอินซ้ำ

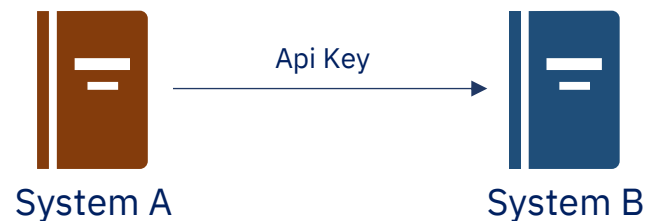


รูปแบบของ Authentication (D1)

การยืนยันตัวตนระหว่างเครื่องกับเครื่อง (Machine-to-Machine / M2M)

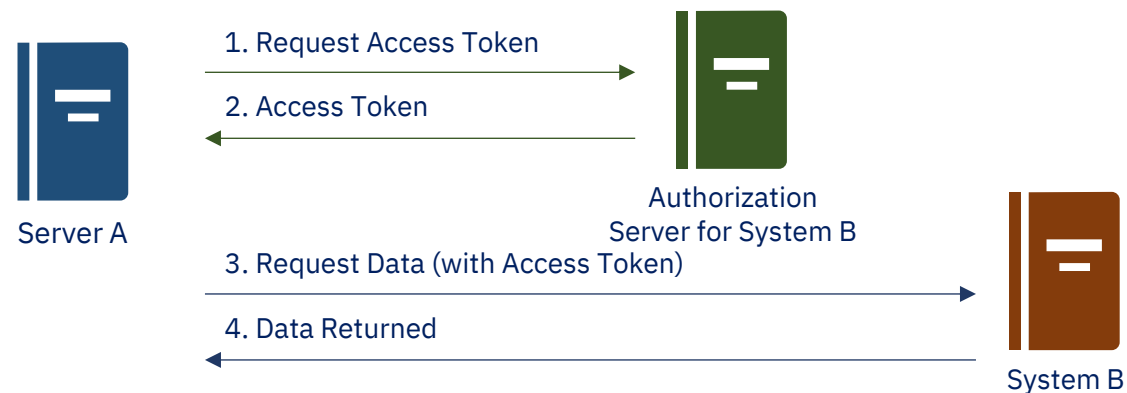
API Key

คีย์สตริงยาวๆ สำหรับระบบภายนอกเรียกใช้งาน API เหมาะกับงานที่ไม่ซับซ้อน



OAuth 2.0 Client Credentials Flow

เซิร์ฟเวอร์ใช้ Client ID + Client Secret ขอ Access Token เพื่อคุยกับอีกเซิร์ฟเวอร์หนึ่ง

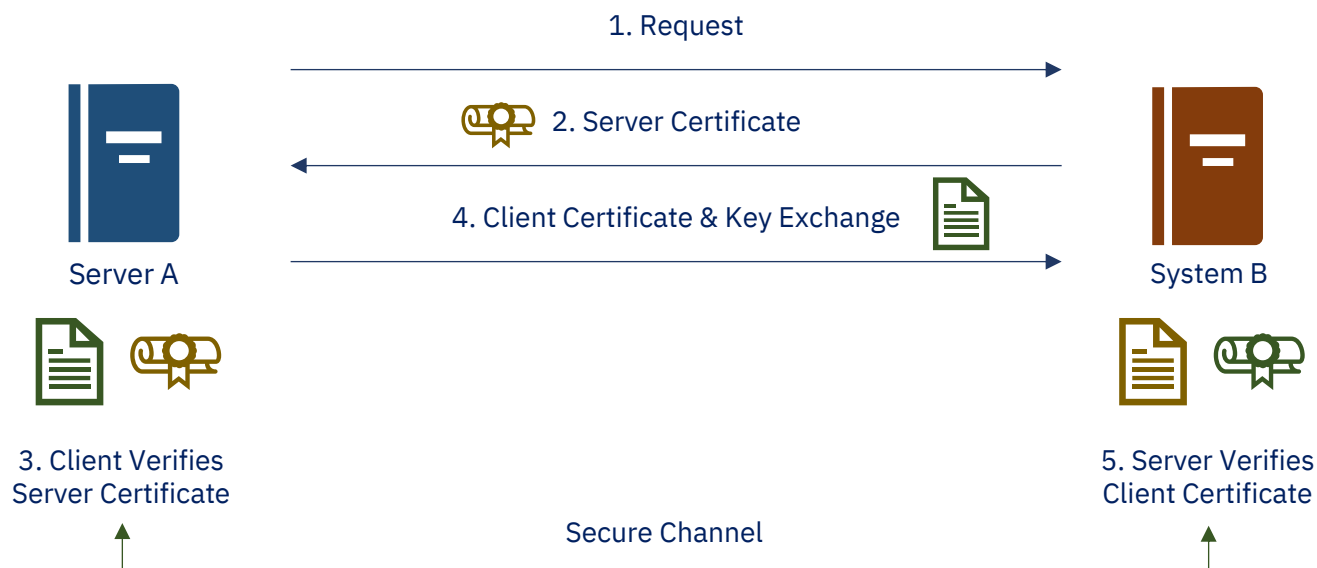


รูปแบบของ Authentication (D2)

การยืนยันตัวตนระหว่างเครื่องกับเครื่อง
(Machine-to-Machine / M2M)

Mutual TLS (mTLS)

ทั้งเซิร์ฟเวอร์และไคลเอนต์ต่างมี Digital Certificate ตรวจสอบความถูกต้องซึ่งกันและกันผ่านระดับเครือข่าย



รูปแบบของ Authorization

RBAC

(Role-Based Access Control)

ควบคุมตามบทบาท

ผู้ใช้ -> กำหนด Role -> Role ผูกกับ Permissions

ระบบทั่วไปที่มีตำแหน่งหน้าที่งานชัดเจน ไม่ซับซ้อน

		Permission		
		ขายเมนู	รายงาน ยอดขาย	จัดการ ข้อมูลร้าน
Role	Manager	✓	✓	✓
	Staff	✓	✓	

รูปแบบของ Authorization

DAC (Discretionary Access Control) / ACL (Access Control List)

เจ้าของข้อมูล (Owner) เป็นผู้กำหนดสิทธิ์เอง
ว่าใครเข้าถึงได้บ้าง

MAC (Mandatory Access Control)

ระบบส่วนกลางเป็นผู้บังคับใช้สิทธิ์อย่าง
เด็ดขาดตามชั้นความลับ ผู้ใช้ทั่วไปไม่สามารถ
ส่งต่อสิทธิ์เองได้

		Permission		
		ขายเมนู	รายงาน ยอดขาย	จัดการ ข้อมูลร้าน
Role	Manager	✓	✓	✓
	Staff	✓	✓	
User	Staff	ขายเมนู	รายงาน ยอดขาย	จัดการ ข้อมูลร้าน
	user: user01		✓	✓
	user: user02	✓		

รูปแบบของ Authorization

ABAC (Attribute-Based Access Control) ควบคุมตามคุณลักษณะและบริบท

ตรวจสอบสิทธิ์โดยคำนวณจากหลายตัวแปร
พร้อมกัน:

Subject: ตำแหน่ง, แผนก, สาขา

Resource: ระดับความลับของเอกสาร,
แผนกเจ้าของข้อมูล

Action: อ่าน, เขียน, อนุมัติ, ลบ

Environment: เวลาทำการ, ตำแหน่ง GPS,
เชื่อมต่อจาก IP ภายในบริษัทหรือไม่

ระบบที่มีเงื่อนไขซับซ้อน
หรือ ต้องการความปลอดภัยขั้นสูง

		Permission		
		ขายเมนู	รายงาน ยอดขาย	จัดการ ข้อมูลร้าน
Role	Manager	✓	✓	✓
	Staff	✓	✓	
สาขา		CNX,CEI,DMK		

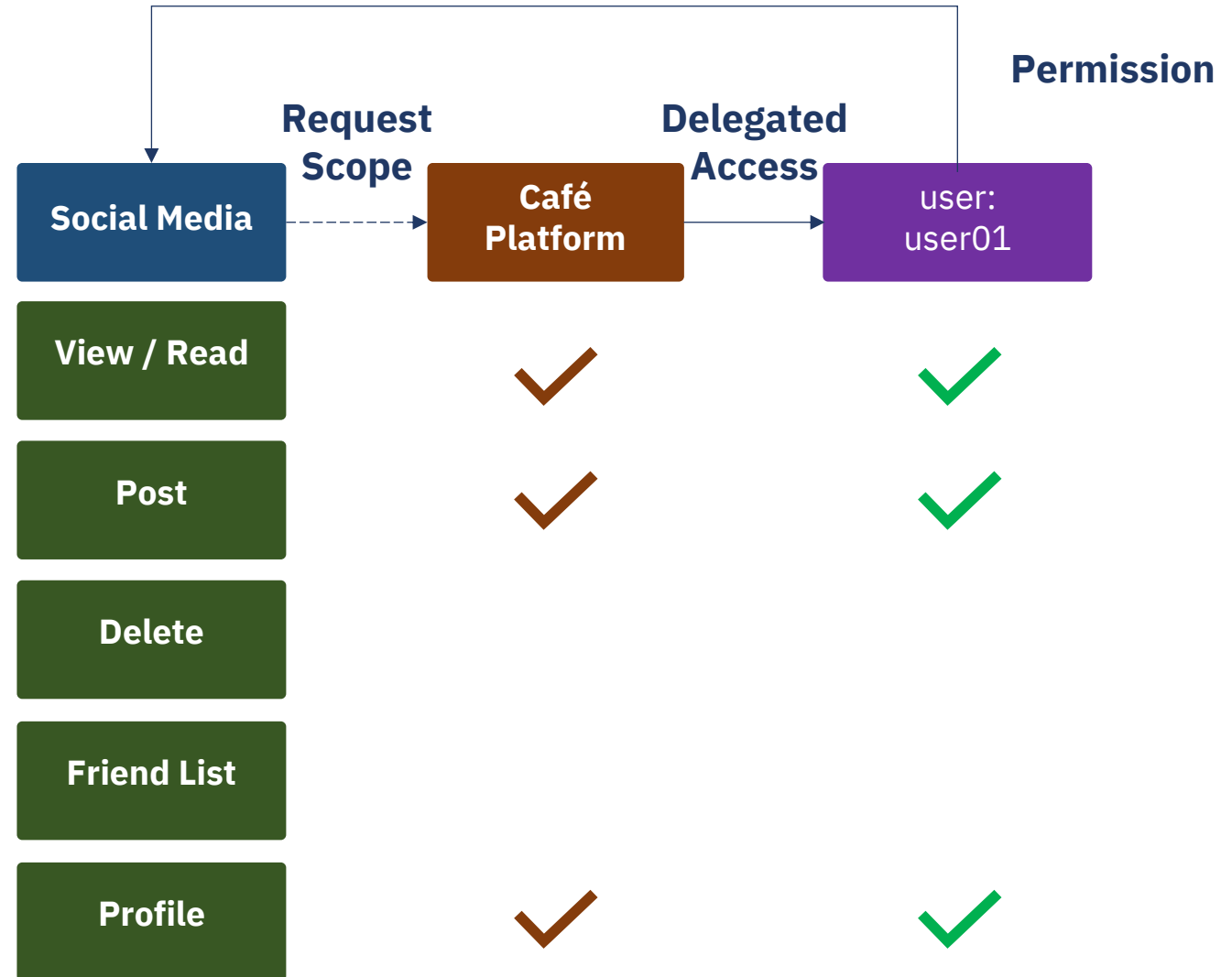
user: manager01	Manager	CNX
user: manager02	Manager	CNX,CEI

รูปแบบของ Authorization

Scope-Based / Delegated Authorization

การมอบอำนาจบางส่วน

ใช้ใน OAuth 2.0 เมื่อผู้ใช้อนุญาตให้แอปภายนอกเข้าถึงข้อมูลบางส่วนแทนตนเอง

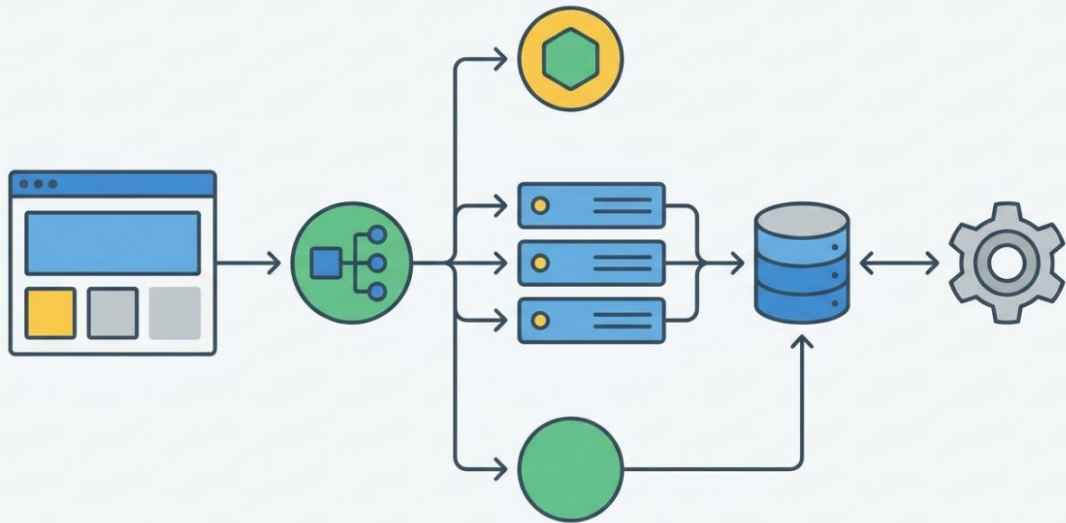


A solid dark blue vertical bar on the left side of the image.

DETAILED DESIGN

Detailed Design

รายละเอียดเชิงลึกว่า "ภายในแต่ละโมดูลหรือฟังก์ชัน ทำงานอย่างไรแบบ step-by-step"



หากไม่มี Detailed Design เมื่อเรานำ Spec ไปให้ AI Coding AI จะ "เดาใจ" ตรรกะธุรกิจเอาเอง เช่น สั่งตัดสต็อกตอนไหน? ถ้าเงินไม่พอจะตอบว่าอะไร? ซึ่งมักจะทำให้เกิดบั๊กในภายหลัง



Detailed Design จะช่วยให้ AI เขียนเงื่อนไข ตรรกะการทำงาน, การเปลี่ยนสถานะ และ Error Handling ตามกฎธุรกิจโดยไม่คิด logic หรือ เดาค้นมาเอง

Detailed Design



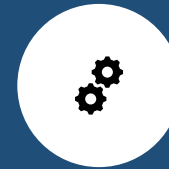
DB Spec

ผังโครงสร้างของฐานข้อมูล
ที่กำหนดว่าข้อมูลจะถูกจัดเก็บ
อย่างไร



API Spec

ข้อความ, ตาราง, หรือ ข้อมูล
ของรายละเอียด การรับ/ส่ง
ข้อมูลระหว่าง Client /
Server ของแต่ละ Endpoint



Security

ข้อความหรือตาราง ที่ระบุ
ข้อกำหนดเฉพาะจุดเพื่ออุด
ช่องโหว่ ป้องกันการโจมตี
หรือการขโมยข้อมูล

Detailed Design More



Pseudo Code

ข้อความแสดงรายละเอียด
อย่างเป็นลำดับขั้นตอน ของ
เงื่อนไขการทำงาน



Exception & Error Handling

ข้อความหรือตารางที่ระบุว่า
เมื่อเกิดข้อผิดพลาด (Edge
Cases/Errors) ระบบต้อง
ตอบกลับอย่างไร และ/หรือ
ต้องจัดการต่ออย่างไร



Business Logic & Validation Rules

ข้อความหรือตาราง ระบุว่า
มีขั้นตอนการทำงาน รวมถึง
เงื่อนไข, สูตรการคำนวณ, ค่า
ขอบเขตการทำงาน อย่างไร
บ้าง

DATABASE SCHEMA DESIGN

DB Schema

ผังโครงสร้างของฐานข้อมูล ที่กำหนดว่าข้อมูลจะถูกจัดเก็บอย่างไร
มีตารางอะไรบ้าง มีคอลัมน์อะไร ข้อมูลประเภทไหน และตารางแต่ละตารางเกี่ยวข้องกันอย่างไร

Conceptual Schema

มองภาพใหญ่ว่าระบบมี Entity / Table อะไรบ้าง และเกี่ยวข้องกันอย่างไร (เช่น ลูกค้า สั่ง ออเดอร์)

Logical Schema

ลงรายละเอียดชื่อคอลัมน์, Primary Key (PK), และ Foreign Key (FK)

Physical Schema

ระบุรายละเอียดเชิงเทคนิคลงไปในฐานข้อมูลจริง เช่น Data Type (VARCHAR(255), BIGINT), Indexing, Partitioning, Storage Engine

Entity-Relationship (ERD)

Table / Entity

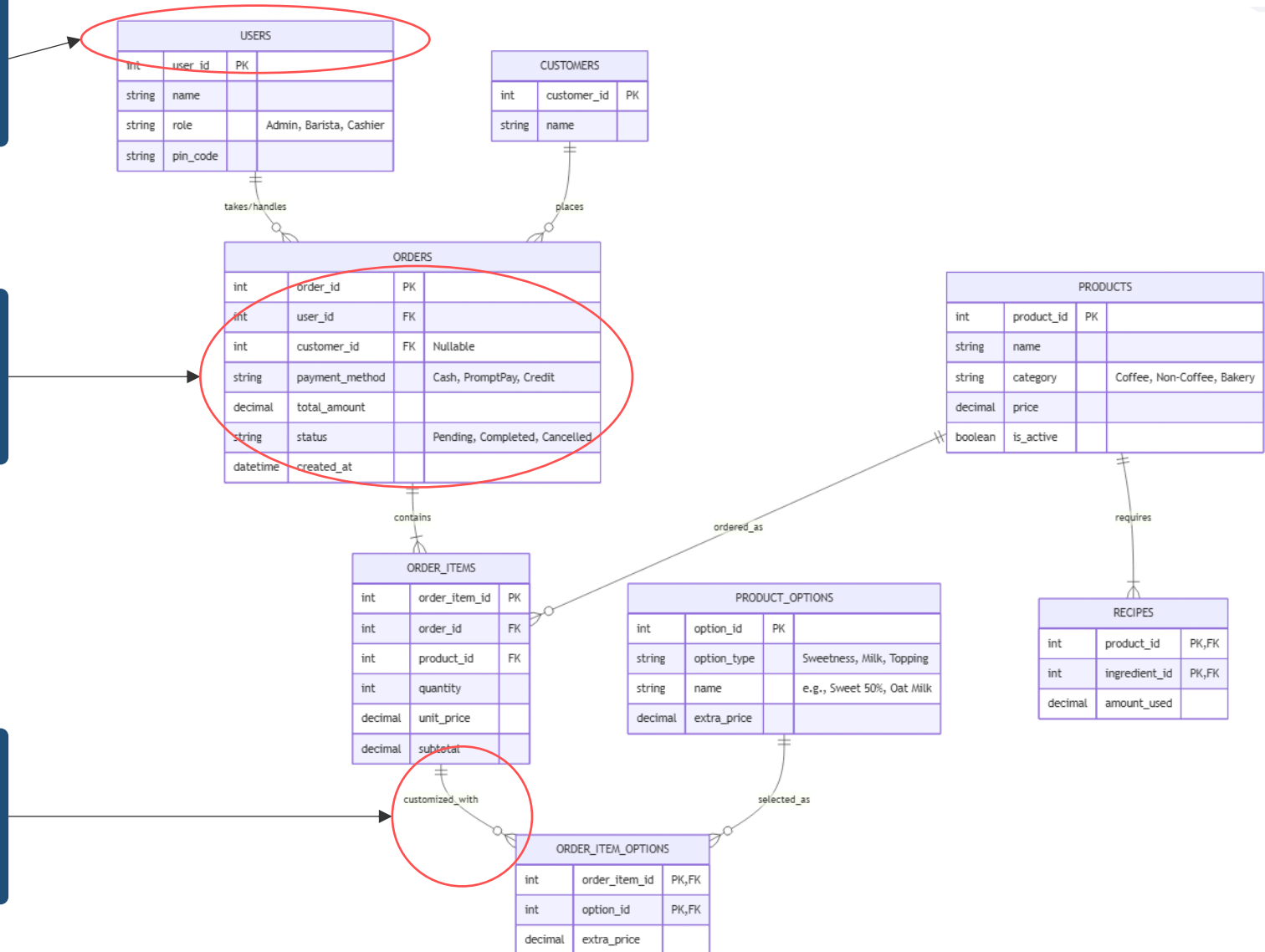
สิ่งที่เราต้องการเก็บข้อมูล

Field / Attribute

หัวข้อรายละเอียดของสิ่งนั้นๆ

Relationship

เส้นเชื่อมโยงบอกว่า ข้อมูลใน 2 ตาราง เกี่ยวข้องกันอย่างไร



SQL vs NoSQL



SQL (Structured Query Language) หรือ RDBMS (Relational Database)

ฐานข้อมูลเชิงสัมพันธ์ที่เก็บข้อมูลในรูปแบบ "ตาราง" (Table) ซึ่งประกอบด้วยแถว (Row) และคอลัมน์ (Column)

- **Fixed Schema:** ต้องกำหนดโครงสร้าง ให้ชัดเจนก่อน
- **ACID Compliance:** เน้นความถูกต้อง, ความน่าเชื่อถือ, ความปลอดภัย, หากเกิดข้อผิดพลาด สามารถย้อนกลับได้
- **Complex Queries:** เด่นเรื่องการเชื่อมโยงข้อมูลข้ามตาราง เพื่อดึงข้อมูลที่มีความซับซ้อน



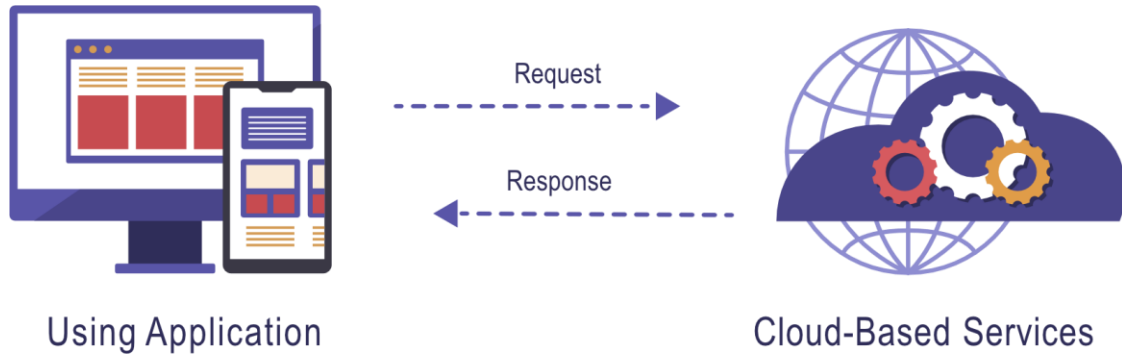
NoSQL (Not Only SQL)

ฐานข้อมูลที่ไม่ใช้โครงสร้างตารางแบบดั้งเดิม ออกแบบมาเพื่อรองรับข้อมูลที่มีปริมาณมหาศาล (Big Data) ข้อมูลที่ไม่มีโครงสร้างแน่นอน และเน้นความเร็วในการอ่าน-เขียนข้อมูล

- **Document Database:** เก็บข้อมูลเป็นเอกสารรูปแบบ JSON/BSON
- **Key-Value Store:** เก็บเป็นคู่ Key และ Value
- **Wide-Column Store:** เก็บข้อมูลแยกตามคอลัมน์
- **Graph Database:** เก็บข้อมูลเป็น Node และ Relationship

API SPEC

API & REST API

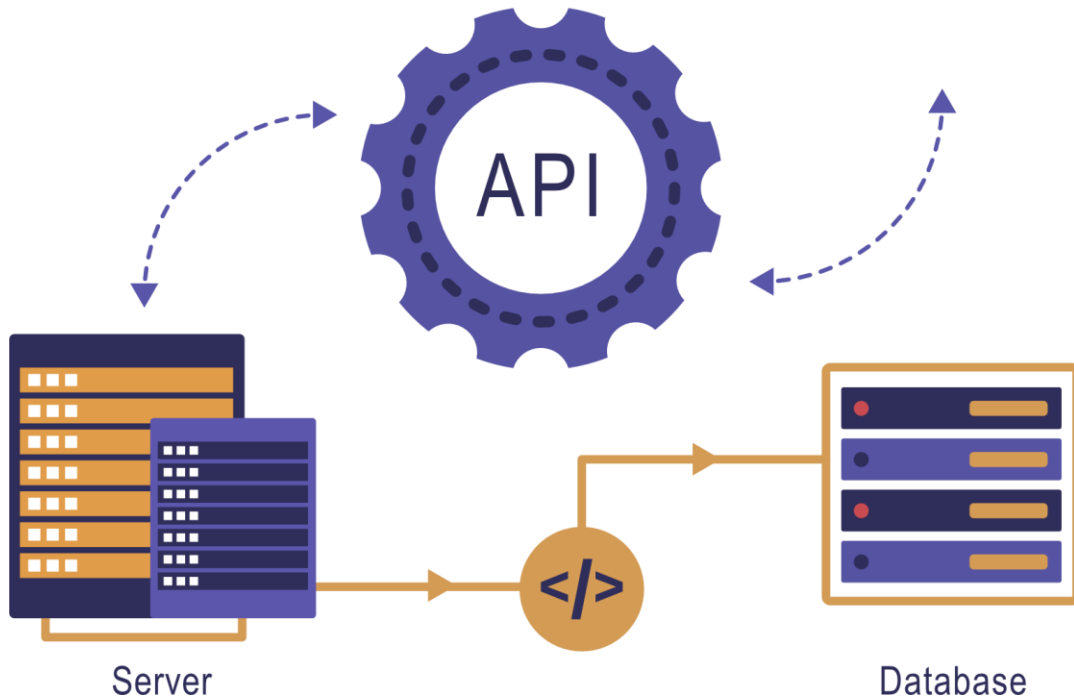


API (Application Programming Interface)

คือ ตัวกลาง ที่ทำหน้าที่เชื่อมต่อและส่งข้อมูลระหว่างโปรแกรมหรือแอปพลิเคชันต่างๆ ให้คุยกันรู้เรื่อง

REST (Representational State Transfer API)

คือ "ภาษาและข้อตกลงมาตรฐาน" ที่ทำให้ระบบต่างระบบกันสามารถรับ-ส่งข้อมูลกันได้ (เช่น หน้าเว็บ UI บน Browser คุยกับ Database ที่อยู่บน Server หลังบ้าน)



API REQUEST



เรา

น้องครับ
กระเพาะไข่ดาว เผ็ดมาก
จานนี้

Order



พนักงาน

ได้ค่ะ
กระเพาะไข่ดาว เผ็ดมาก
1 จานนะคะ

Receive order and take
order to kitchen

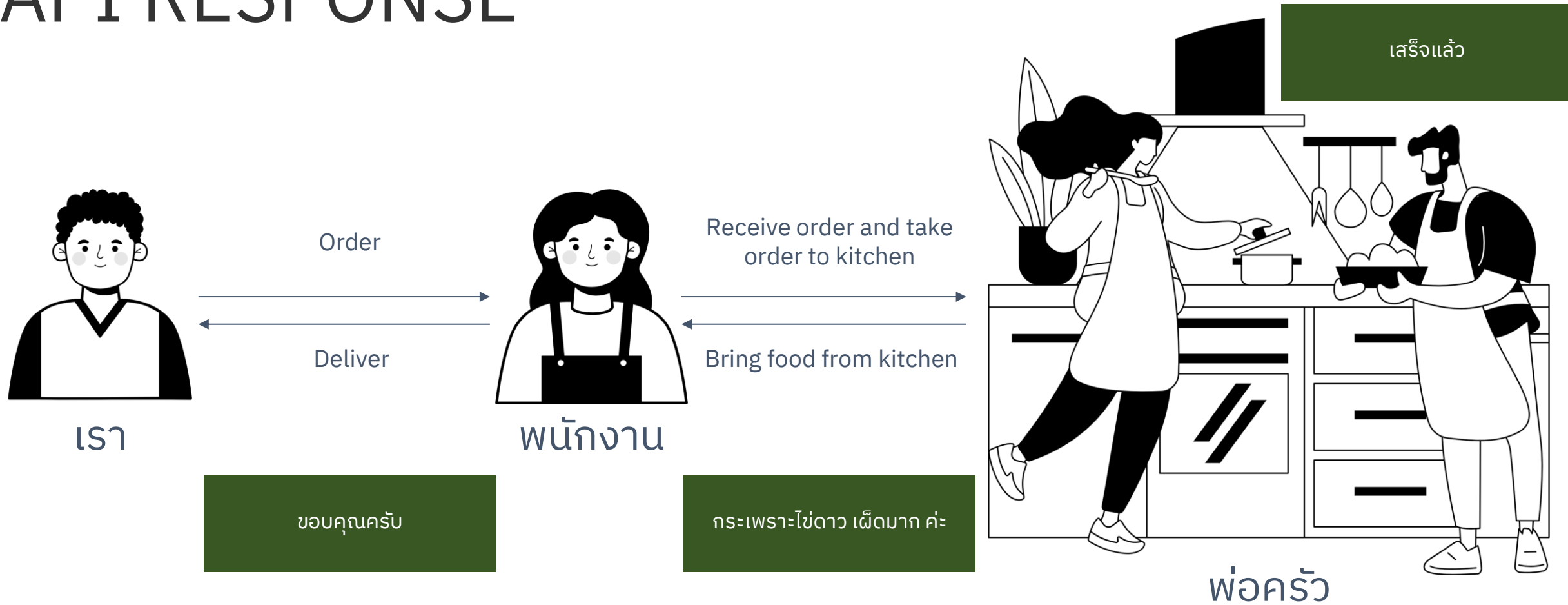


พ่อครัว

Cooking process with food ingredients

- Menu (Fried Basil, Fried Rice)
- Pork (Pork, Chicken, Crispy Pork)
- Egg (Y/N)
- Spicy Level (Low, Medium, High)

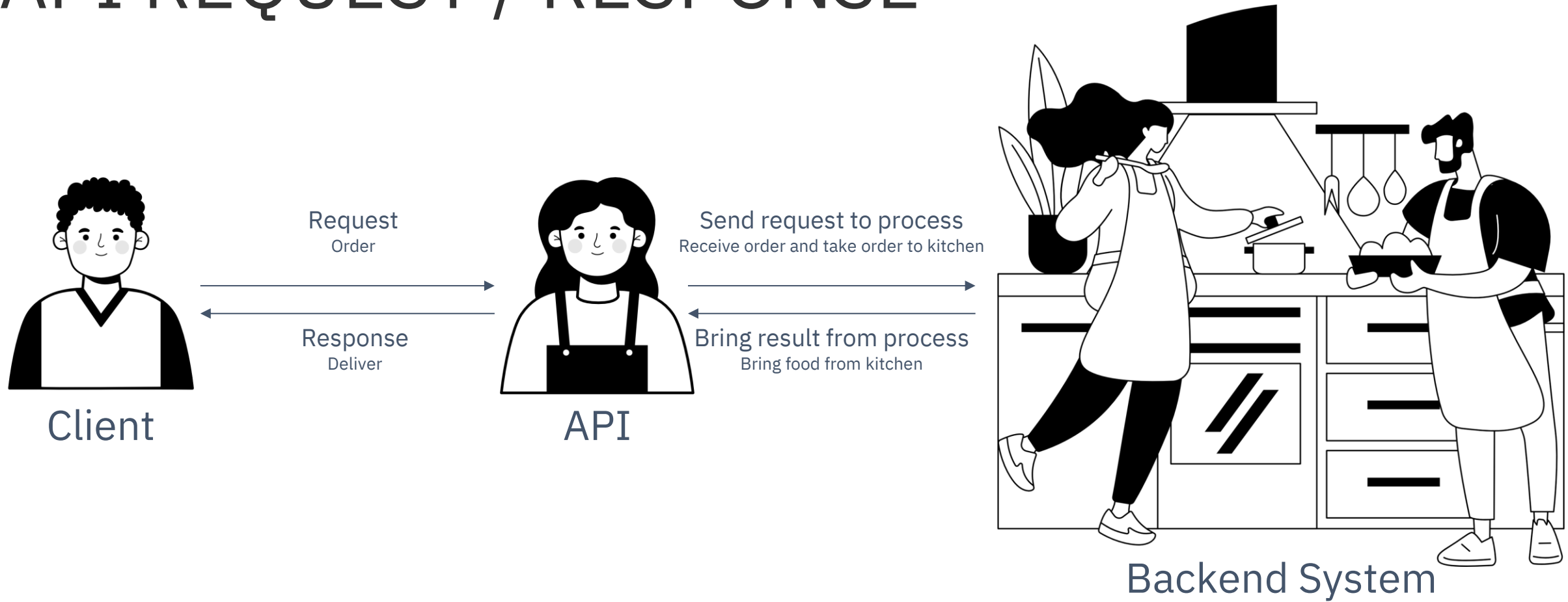
API RESPONSE



Cooking process with food ingredients

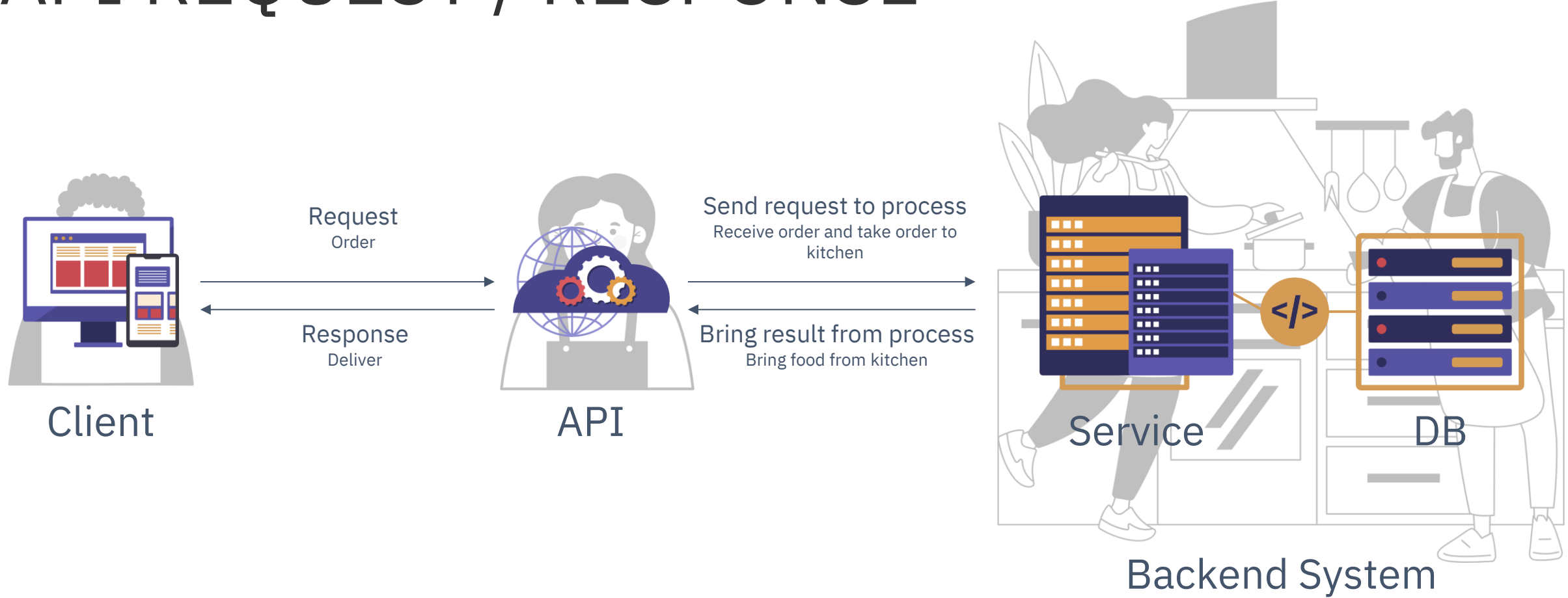
- Menu (Fried Basil, Fried Rice)
- Pork (Pork, Chicken, Crispy Pork)
- Egg (Y/N)
- Spicy Level (Low, Medium, High)

API REQUEST / RESPONSE



- Cooking process with food ingredients - endpoint
- Menu (Fried Basil, Fried Rice) - parameter
 - Pork (Pork, Chicken, Crispy Pork) - parameter
 - Egg (Y/N) - parameter
 - Spicy Level (Low, Medium, High) - parameter

API REQUEST / RESPONSE



- Cooking process with food ingredients - endpoint
- Menu (Fried Basil, Fried Rice) - parameter
 - Pork (Pork, Chicken, Crispy Pork) - parameter
 - Egg (Y/N) - parameter
 - Spicy Level (Low, Medium, High) - parameter

REST API

ตัวอย่าง Request เพื่อสร้างเมนู

Request Payload / Data & Header:

```
{ "name": "Black Coffee", "price": 30.00 }
```

HTTP Request

GET

POST

PUT /
PATCH

DELETE



Client



HTTP Response

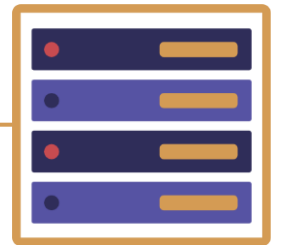
(Status Code, JSON/XML Data)



API



Service



DB

Status: 200 OK

Response Payload / Data:

```
{ "id": 1, "name": "Black Coffee", "price": 30.00 }
```


HTTP Methods

GET

ขอดู/ดึงข้อมูล: เหมือนการเปิดดูเมนู หรือดูรายการสินค้า
(ไม่มีการแก้ไขข้อมูลใน DB)
GET /api/v1/menu-items
GET /api/v1/orders/101

POST

สร้างข้อมูลใหม่: เหมือนการกดสั่งออเดอร์สั่งอาหาร
หรือกดสมัครสมาชิก
POST /api/v1/orders

PUT /
PATCH

แก้ไขข้อมูลเดิม: เหมือนการเปลี่ยนเบอร์โทรศัพท์
หรืออัปเดตสถานะออเดอร์
PUT /api/v1/orders/101

DELETE

ลบข้อมูล: เหมือนการยกเลิกรายการ หรือลบสินค้าออกจากตะกร้า
DELETE /api/v1/orders/101

HTTP Status Codes

200 OK
สำเร็จ

201 Created
สำเร็จ

400 Bad Request
ส่งข้อมูลมาไม่ถูกต้อง

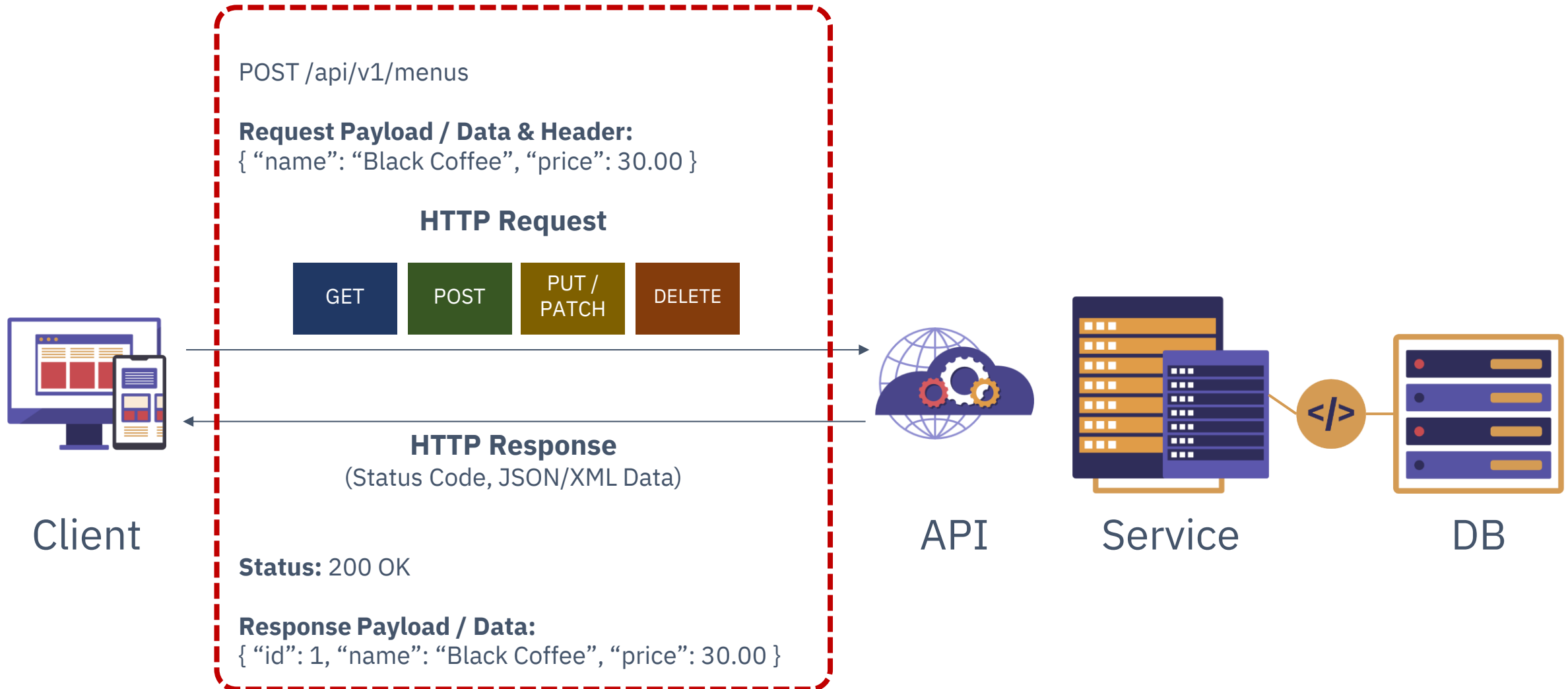
401 Unauthorized
ไม่มีสิทธิ์เข้าถึง

404 Not Found
หาข้อมูลไม่เจอ

500 Internal Server Error
ระบบหลังบ้านมีปัญหา

API Spec

เอกสารที่ระบุรายละเอียด Endpoint, Method, Request/Response ตัวอย่าง และ Validation Rules ทั้งหมดไว้ล่วงหน้า



โครงสร้างเอกสาร API Spec

เอกสารที่ระบุรายละเอียด Endpoint, Method, Request/Response ตัวอย่าง และ Validation Rules ทั้งหมดไว้ล่วงหน้า

📌 API Specification: Coffee Shop Ordering System

- Version: 1.0.0
- Base URL: <https://api.coffeeshop.com/v1>
- Format: JSON
- Traceability: Linked to [[01-requirements/backlog#US-01]] & [[02-design/01-prototypes/index.html]]

📄 Summary of Endpoints

Method	Endpoint	Description	Auth Required
GET	/menu-items	ดึงรายการเมนูกาแฟ/เครื่องดื่มทั้งหมด	No
POST	/orders	สร้างออเดอร์สิ่งใหม่จากโต๊ะ	No
GET	/orders/{order_id}	ดึงข้อมูลและสถานะออเดอร์ตาม ID	No
PATCH	/orders/{order_id}/status	อัปเดตสถานะออเดอร์ (สำหรับพนักงาน)	Yes (Admin)

2. Create New Order

สร้างออเดอร์สิ่งใหม่เมื่อลูกค้าสั่งโต๊ะและสั่งอาหารผ่านหน้าเว็บ UI

- Method: POST
- Endpoint: /orders
- Authentication: None

Request Body (application/json)

```
{
  "table_number": 5,
  "items": [
    {
      "menu_id": "M-001",
      "quantity": 2,
      "sweetness_level": "50%",
      "note": "แยกน้ำแข็ง"
    }
  ]
}
```

Validation Rules (เงื่อนไขการตรวจสอบ)

- table_number: ต้องเป็นตัวเลขจำนวนเต็ม และอยู่ระหว่าง 1 - 30
- items: ต้องมีอย่างน้อย 1 รายการ (array_length >= 1)
- quantity: ต้องมากกว่าหรือเท่ากับ 1

Response Example Success (201 Created)

```
{
  "status": "success",
  "message": "สร้างออเดอร์สำเร็จ",
  "data": {
    "order_id": "ORD-20260813-001",
    "table_number": 5,
    "total_price": 120.00,
    "order_status": "PENDING_PAYMENT",
    "created_at": "2026-08-13T16:00:00Z"
  }
}
```

Response Example Error (400 Bad Request)

กรณีส่งข้อมูลไม่ถูกต้อง เช่น ระบุเลขโต๊ะ หรือรายการสินค้าไม่มีในระบบ

```
{
  "status": "error",
  "error_code": "INVALID_INPUT",
  "message": "โปรดระบุหมายเลขโต๊ะให้ถูกต้อง",
  "details": [
    {
      "field": "table_number",
      "issue": "table_number is required"
    }
  ]
}
```

Response & Status Code

Validation Rules & Business Logic

Definition & Description

Endpoints & Operations

Authentication & Security

Request Parameters

A solid dark blue vertical bar is positioned on the left side of the image, extending from the top to the bottom.

ADDITIONAL DETAILED DESIGN

Additional Detailed Design

Exception & Error Handling

กรณีข้อผิดพลาด การจัดการปัญหาที่อาจเกิดขึ้น (ดักจับ) หรือสถานการณ์ที่ไม่ปกติ และระบุว่าระบบต้องรับมือหรือแจ้งเตือนผู้ใช้อย่างไร

สิ่งที่ควรมี

- Scenario (สถานการณ์ที่ผิดพลาด): เช่น เน็ตหลุด, ข้อมูลผิด, คนแย่งกันซื้อ
- Expected Behavior (การจัดการของระบบ): ระบบจะทำงานอย่างไร
- Error Message / Status Code: ข้อความที่ตอบกลับไปหาผู้ใช้

ตัวอย่าง:

1. Case 1: สิ้นค้าหมดกระถันหัน

(Race Condition - คนกดซื้อพร้อมกัน)

- Handling: ระบบต้องจับ (Catch) Error ตอนตัดสต็อก ไม่สร้างออเดอร์
- Response: ส่ง HTTP 409 Conflict และส่ง Message แจ้ง Frontend ว่า "ขอภัย สินค้า X หมดสต็อก"

2. Case 2: ระบบ Payment Gateway ล่ม (3rd Party Error)

- Handling: บันทึกออเดอร์เป็นสถานะ payment_failed
- Response: ส่ง HTTP 503 Service Unavailable แจ้งว่า "ระบบชำระเงินขัดข้องชั่วคราว กรุณาลองใหม่ในอีก 5 นาที"

Additional Detailed Design

Security

ข้อกำหนดเฉพาะจุดเพื่ออุดช่องโหว่ ป้องกันการโจมตี หรือการขโมยข้อมูล

สิ่งที่ควรมี

- Input Validation & Sanitization: การกรองข้อมูลก่อนรับเข้า DB เพื่อป้องกัน SQL Injection หรือ XSS
- Rate Limiting: การจำกัดจำนวนครั้งในการยิง API (ป้องกัน Bot/Spam)
- Data Masking / Encryption: การเข้ารหัสข้อมูลส่วนตัว หรือเซ็นเซอร์ข้อมูลก่อนส่งออก

ตัวอย่าง (Security สำหรับระบบ Order):

1. Validate Payload ทุกครั้ง (เช่น จำนวนสินค้า qty ต้องเป็นตัวเลขและมากกว่า 0 เท่านั้น ห้ามใส่ค่าติดลบ)
2. จำกัดการเรียก API POST /api/v1/orders ได้ไม่เกิน 5 ครั้งต่อ 1 นาที ต่อ 1 User (ป้องกันการสร้างออเดอร์รัวๆ)
3. ข้อมูลบัตรเครดิตที่รับมา ห้ามบันทึกลง Database เด็ดขาด (ส่งตรงไปให้ Payment Gateway)
4. เมื่อดึงข้อมูล Order กลับไปแสดงผล (API Response) ต้องทำการ Masking เลขบัญชี (เช่น ส่งกลับไปแค่ ***-***-1234)

Additional Detailed Design

Business Logic & Validation Rules

ตรรกะขั้นตอนการทำงาน รวมถึงเงื่อนไข, สูตรการคำนวณ, ค่าขอบเขตการทำงาน อย่างไรก็ตาม

สิ่งที่ควรมี

- ลำดับขั้นตอน (Step-by-step description) หรือเขียนเป็น Pseudocode
- เงื่อนไขทางธุรกิจ (Business Rules) เช่น "ห้ามซื้อเกิน 10 ชิ้นต่อคน"
- Sequence Diagram หรือ Flowchart (แนะนำให้ใช้ เพื่อช่วยให้ผู้อ่านง่าย)

Pseudo Code

รายละเอียด อย่างเป็นลำดับขั้นตอน ของเงื่อนไขการทำงาน เป็นการเขียนเค้าโครงของโปรแกรมหรืออัลกอริทึมโดยใช้ภาษามนุษย์ทั่วไป ผสมกับโครงสร้างของภาษาคอมพิวเตอร์

ตัวอย่าง (ขั้นตอนการทำงานของ API Create Order):

1. รับค่า cart_id และตรวจสอบว่า Token ของผู้ใช้ถูกต้องหรือไม่
2. ดึงข้อมูลสินค้าใน Cart จาก Database
3. Check Stock: ตรวจสอบว่าสินค้าในสต็อกมีเพียงพอหรือไม่
4. Calculate: คำนวณราคาสุทธิ (หักส่วนลด, บวกค่าส่ง)
5. Transaction: ตัดสต็อกสินค้า และ สร้างข้อมูลลงตาราง Orders (หากทำข้อใดข้อหนึ่งพลาด ให้ Rollback กลับทั้งหมด)
6. ส่งข้อมูลการชำระเงินไปที่ Payment Gateway
7. ตอบกลับ order_id ให้ Frontend

NON- FUNCTIONAL REQUIREMENT (NFR)

Non-Functional Requirement (NFR)

ความต้องการที่ไม่ใช่ฟังก์ชันการทำงานหลัก (เช่น ความเร็ว, การรองรับผู้ใช้งานจำนวนมาก, ความน่าเชื่อถือของระบบ)

Functional Requirement (FR)

ระบบ "ทำอะไรได้บ้าง" (What it does) เช่น ระบบสามารถล็อกอินได้, กดสั่งซื้อสินค้าได้, ออกรายงาน PDF ได้

Non-Functional Requirement (NFR)

ระบบ "ทำสิ่งนั้นได้ดีแค่ไหน" (How well it does) หรือเปรียบเสมือน "คุณภาพ" ของระบบ เช่น ล็อกอินได้ภายใน 1 วินาที, ระบบไม่ล่มแม้มีคนสั่งซื้อพร้อมกันพันคน

หมายเหตุ: บางเรื่องไม่จำเป็นต้องมีตั้งแต่วันแรกก็ได้ แต่ควรระวัง ถ้าไม่มีอาจจะเกิดปัญหาเรื่อง Technical Debt คือ เมื่อเพิ่มเติมภายหลัง อาจจะต้องใช้ระยะเวลาในการแก้ไขสูงกว่า

แต่ในความเป็นจริง ก็จะมีหลายๆ ครั้ง ที่ค่อยๆ เพิ่มเข้ามาภายหลัง เนื่องจากการพัฒนาแบบวนซ้ำ (Iteration) แต่ควรเพิ่มก่อนที่จะเริ่มพัฒนาใน feature หรือ ส่วนนั้นๆ

ตัวอย่างเปรียบเทียบ (Analogy): การซื้อรถยนต์

- **FR:** รถต้องมีพวงมาลัย, มีแอร์, ขับเดินทาง่ายหลังได้ (ฟังก์ชันพื้นฐาน)
- **NFR:** รถต้องทำความเร็วสูงสุดได้ 200 กม./ชม., ประหยัดน้ำมัน 20 กม./ลิตร, ได้รับมาตรฐานความปลอดภัยระดับ 5 ดาว (คุณภาพของฟังก์ชัน)

ข้อควรระวัง

- **แบบที่ผิด (ตีความลำบาก):** ระบบต้องทำงานเร็วมาก, ระบบต้องมีความปลอดภัยสูง
- **แบบที่ถูกต้อง (ระบุตัวเลขชัดเจน):** ระบบต้องประมวลผลการค้นหาข้อมูล 1 ล้าน Record ได้ภายใน 3 วินาที, ระบบต้องบังคับให้ผู้ใช้งานเปลี่ยนรหัสทุกๆ 90 วัน

Non-Functional Requirement (NFR)

Performance
(ประสิทธิภาพ)

Scalability
(การขยายตัว)

Availability & Reliability
(ความพร้อมและเสถียรภาพ)

Security
(ความปลอดภัย)

Usability
(ความง่ายในการใช้งาน)

Maintainability
(การบำรุงรักษา)

ตัวอย่างหมวดหมู่ของ NFR ที่ใช้บ่อย

- **Performance:** ความเร็วในการประมวลผล เช่น หน้าเว็บต้องโหลดเสร็จภายใน 2 วินาที
- **Scalability:** การรองรับการเติบโต เช่น ระบบต้องรองรับผู้ใช้งานพร้อมกัน (Concurrent Users) ได้ 10,000 คนโดยไม่หน่วง
- **Availability & Reliability:** ระบบห้ามล่ม เช่น ต้องมี Uptime 99.9% (อนุญาตให้ระบบล่มได้ไม่เกิน 8.7 ชั่วโมงต่อปี)
- **Security:** การปกป้องข้อมูล เช่น รหัสผ่านต้องถูกเข้ารหัสแบบ Bcrypt เสมอ, ข้อมูลบัตรเครดิตต้องถูกมาस्क
- **Usability:** ประสิทธิภาพผู้ใช้ เช่น ผู้ใช้งานทั่วไปต้องสามารถทำการสั่งซื้อเสร็จได้ภายใน 3 คลิก
- **Maintainability:** การดูแลหลังบ้าน เช่น ระบบต้องมีการเขียน Log บันทึกทุก Error เพื่อให้ทีมโปรแกรมเมอร์ตามแก้ปัญหาได้ทันที

TECHNOLOGY SELECTION

Technology Selection

แนวทางในการเลือกเทคโนโลยี รวมถึงปัจจัยที่เกี่ยวข้อง

ความเหมาะสมกับ Business	ตอบโจทย์ requirement	ความเร็ว ในการส่งมอบ	ต้นทุน และ งบประมาณ	ความคุ้มค่า กับการลงทุน
ด้านเทคนิค	Non-Functional Requirement	การรองรับการเติบโต, ความเร็ว, ความเสถียร, มาตรฐานความปลอดภัย, การจัดการช่องโหว่, ความเข้ากันได้กับระบบเดิม และเทคโนโลยีที่ใช้, การปรับแต่ง หรือขยาย Feature ในอนาคต		
ด้านเทคโนโลยี ในมุมมองของระบบนิเวศ และความยั่งยืน	Maturity / Stability	Community / Documentation	Third-Party Integration	Vendor Support
ทีมงาน	ความเชี่ยวชาญ ของทีม	การเรียนรู้	การหาคน ที่มีทักษะนี้ในตลาด	
ต้นทุนระยะยาว	ค่า License / Subscription	ค่า Infrastructure (Server / Cloud)	ค่าบำรุงรักษา	ค่าใช้จ่ายจากการ migrate ในอนาคต
ความเสี่ยง	ผูกติดกับผู้ให้บริการ มากเกินไปหรือไม่	อนาคตของ เทคโนโลยี	สอดคล้องกับ กฎหมาย หรือ มาตรฐานที่ต้องการ	ข้อมูลถูกเก็บที่ไหน

ขั้นตอนในการเลือก Technology เบื้องต้น

2. Research & Shortlist

สำรวจ และเลือกเทคโนโลยี ที่เป็นไปได้ เพื่อนำมาเปรียบเทียบ

4. Proof of Concept (POC)

ทดสอบด้วย Feature สำคัญ เพื่อดูว่าติดปัญหาในการใช้งานจริงหรือไม่

1. Define Requirement

กำหนดความต้องการ ทั้ง Functional และ Non-Functional Requirement รวมถึงข้อจำกัดต่างๆ

3. ทำตารางเปรียบเทียบ และบันทึกข้อมูล

วิเคราะห์ด้วยเทคนิคต่างๆ

- Trade-Offs Matrix เปรียบเทียบ ข้อดีข้อเสีย
- Scoring Matrix หรือ Weighted Criteria เปรียบเทียบ

ตัดสินใจและบันทึกเหตุผล

เทคนิคอื่นๆ ที่ใช้นำมาช่วยตัดสินใจได้ เช่น SWOT, Technology Radar

ข้อผิดพลาดที่พบบ่อย Anti-Patterns / Pitfalls

Hype-Driven Decision
(เลือกตามกระแส)

Total Cost of Ownership
มองข้ามต้นทุน

ไม่ได้ทำ Proof of Concept (POC)

Over-Engineering
(เลือกเทคโนโลยีที่ซับซ้อน
เกินความจำเป็น)

ไม่ได้คิดถึง Compliance /
Regulatory ตั้งแต่แรก

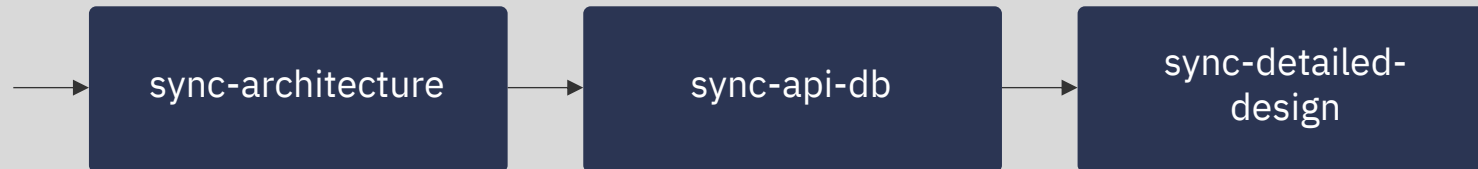
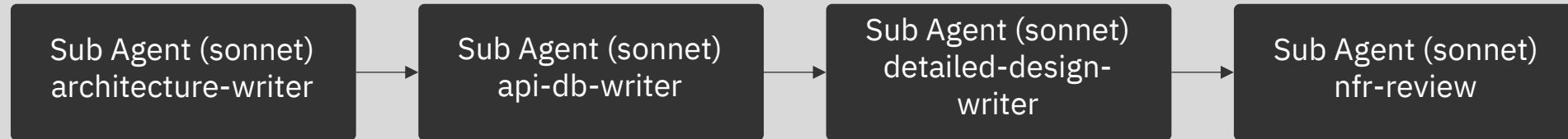
Ignore Exit Strategy
Vendor Lock-in
ผูกติดกับ Vendor / Cloud Provider
รายเดียวมากเกินไป

ยึดติดกับเทคโนโลยีเดิม
เพราะความเคยชิน

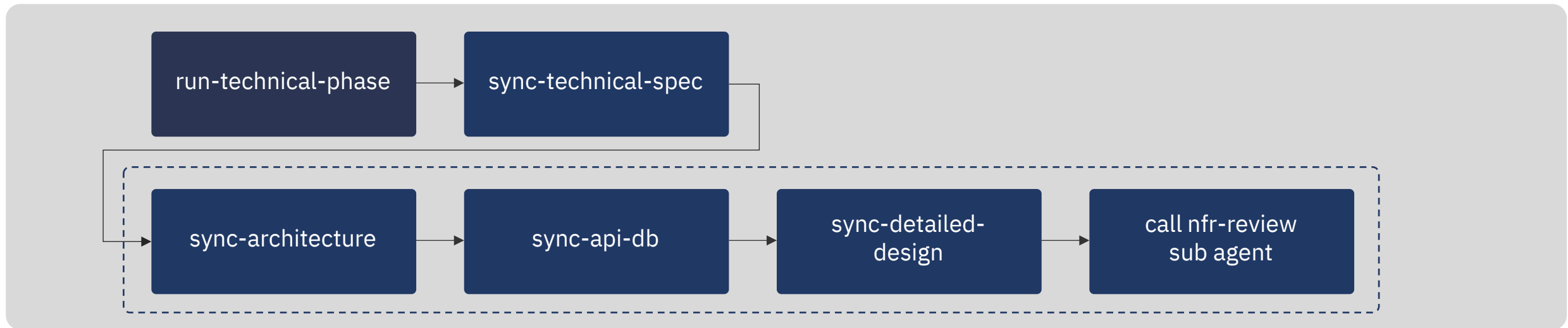
STARTER KIT

แนวคิดของ Starter Kit (Technical)

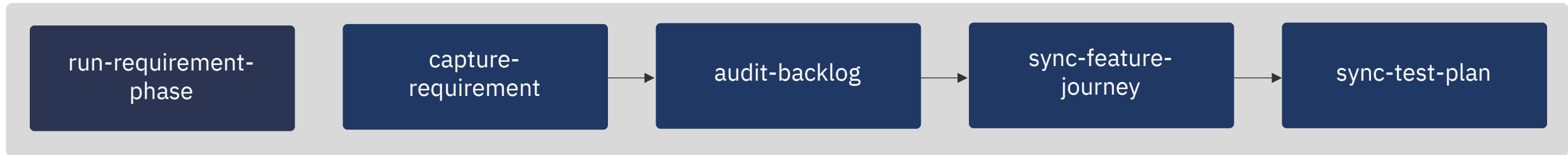
Architecture
API Spec Plan + Task
DB Spec



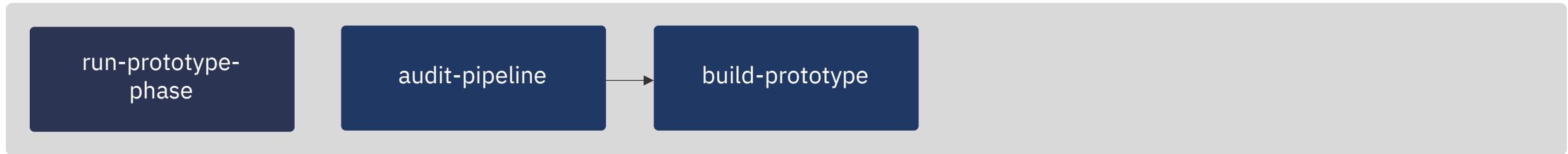
แนวคิดของ Starter Kit (Orchestrator Tech Spec)



แนวคิดของ Starter Kit (Orchestrator Requirement)



แนวคิดของ Starter Kit (Orchestrator Prototype)



RECAP

SDLC and Agile with AI

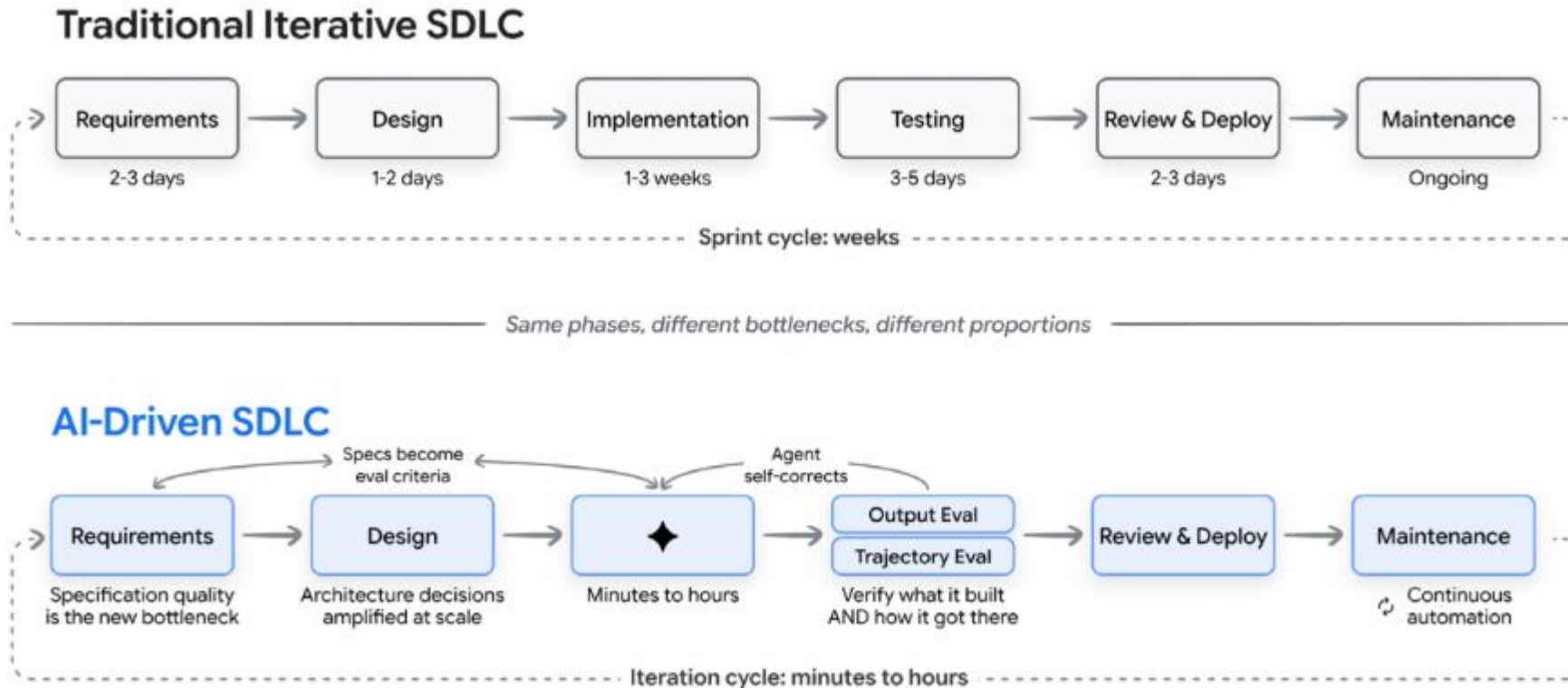
SDLC



Scrum Artifact / Output



New SDLC with Vibe Coding

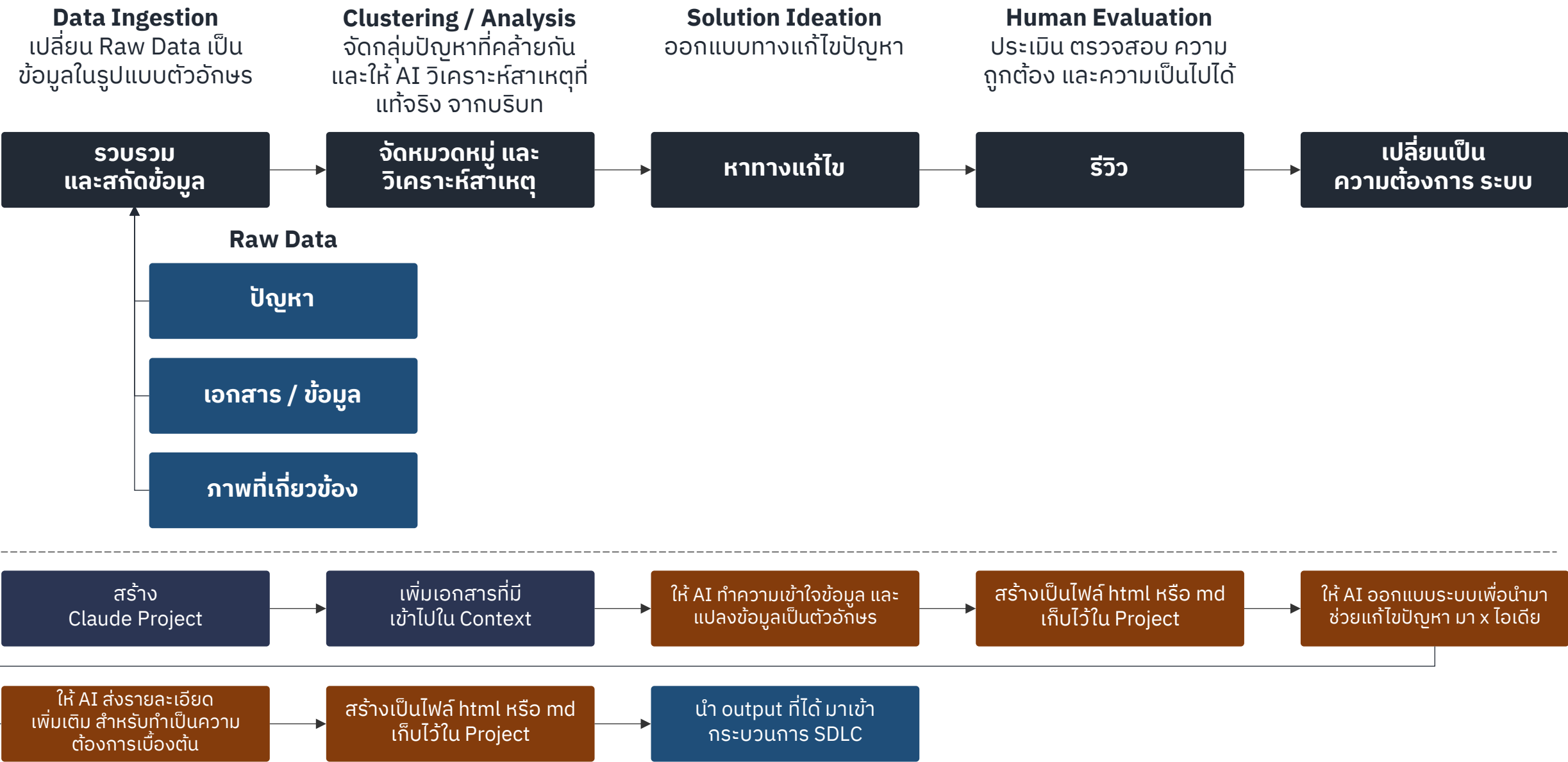


PAIN POINT TO REQUIREMENT

เสริมเรื่องการแปลงโจทย์ที่มี เป็นความต้องการ เพื่อเตรียมพร้อมสำหรับจัดเตรียมเอกสาร

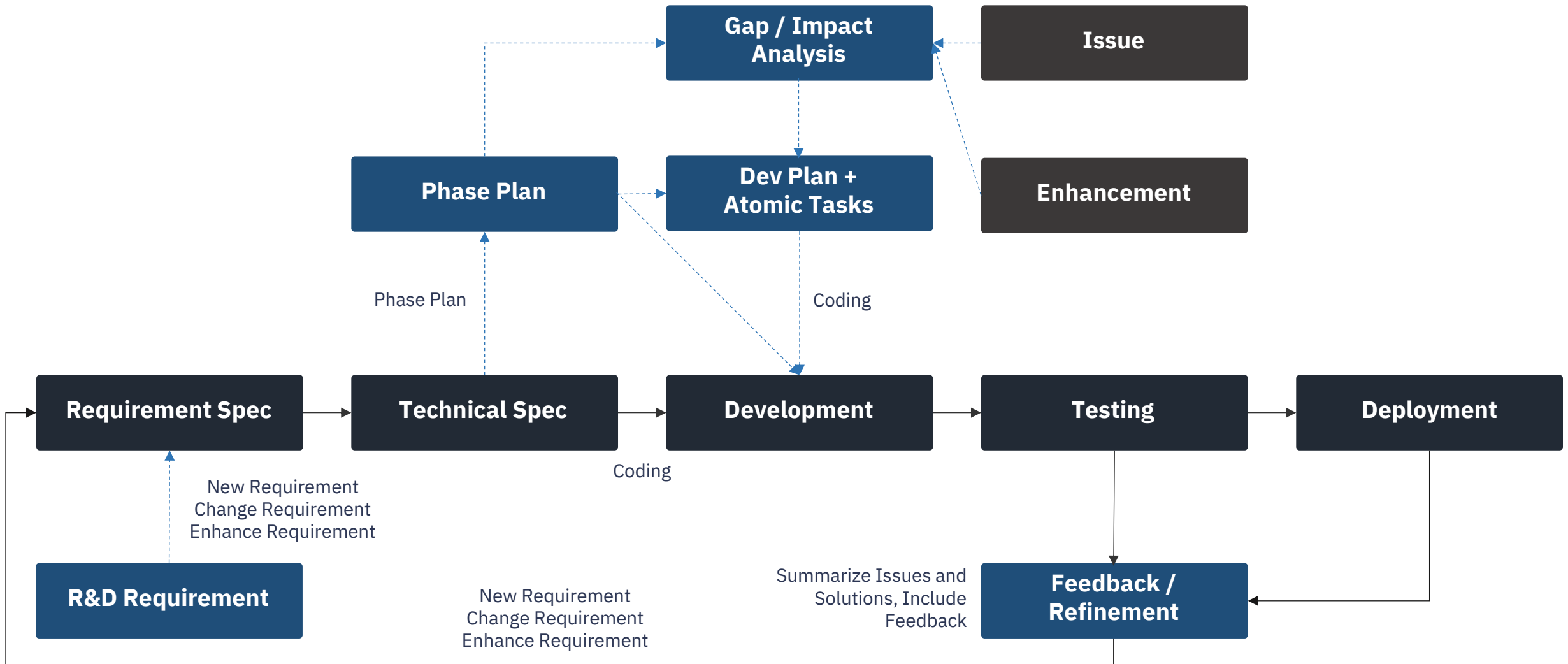
Pain Point to Requirement

สำหรับใครที่มีความต้องการเป็นเอกสาร หรือข้อมูลดิบอื่นๆ



DEVELOPMENT FLOW EXAMPLE

ตัวอย่างภาพรวม Step ในการทำงานในระยะยาว



WORKSHOP

Workshop: สร้าง Technical Spec

โจทย์ 1: จาก Requirement Spec, Product Backlog, Feature List และ User Journey, Interactive Prototype และ Acceptance Criteria, Test Case, และ Test Plan ที่มี
สร้าง Architecture, Database Schema, และ API Spec

ขั้นตอน (ทำงานร่วมกับ AI)

1. สร้าง Sub Agent & Agent Skill สำหรับช่วยในการสร้าง Architecture
2. สร้าง Sub Agent & Agent Skill สำหรับช่วยในการสร้าง DB Schema & API Spec
3. สร้าง Sub Agent & Agent Skill สำหรับช่วยในการสร้าง Detailed Design
4. ปรับปรุง CLAUDE.md
5. เรียกใช้งาน Sub Agent & Agent Skill เพื่อสร้าง Architecture
6. เรียกใช้งาน Sub Agent & Agent Skill เพื่อสร้าง DB Schema & API Spec
7. เรียกใช้งาน Sub Agent & Agent Skill เพื่อสร้าง Detailed Design
8. Commit และ Push เอกสาร ให้เป็นเวอร์ชันล่าสุดอีกครั้ง

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill สำหรับสร้าง High Level Architecture

สร้าง agent และ agent skill

สำหรับสร้าง หรือปรับปรุง เอกสาร high level architecture

ในลักษณะ conceptual ซึ่งยังไม่ผูกมัดกับ technical stack

โดยมีข้อมูล data flow ตาม user journey และรายละเอียดอื่นๆ ที่ควรมี

หากมีส่วนไหนไม่ชัดเจน ให้ถามผู้ใช้งานเพิ่มเติม พร้อมกับมีคำแนะนำ หรือแนวทางให้เลือกอย่างน้อย 3

คำแนะนำ หรือ 3 แนวทาง พร้อมข้อดี ข้อเสีย เพื่อให้ผู้ใช้พิจารณา

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill สำหรับสร้าง Database Schema และ API Spec

สร้าง agent และ agent skill

สำหรับสร้าง หรือปรับปรุง เอกสาร api spec และ database schema หรือ database spec

ในลักษณะ conceptual ซึ่งยังไม่ผูกมัดกับ technical stack

มีข้อมูลรายละเอียดแต่ละตาราง, และ ER Diagram เป็นอย่างน้อย หรือข้อมูลอื่นๆ ตามที่สมควร

หากมีส่วนไหนไม่ชัดเจน ให้ถามผู้ใช้งานเพิ่มเติม พร้อมกับมีคำแนะนำ หรือแนวทางให้เลือกอย่างน้อย 3

คำแนะนำ หรือ 3 แนวทาง พร้อมข้อดี ข้อเสีย เพื่อให้ผู้ใช้พิจารณา

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill สำหรับสร้าง Detailed Design

สร้าง agent และ agent skill

สำหรับสร้าง หรือปรับปรุง detailed design

ในลักษณะของ conceptual design มีข้อมูล sequence flow เป็นอย่างน้อย

หากมีส่วนไหนไม่ชัดเจน ให้ถามผู้ใช้งานเพิ่มเติม พร้อมกับมีคำแนะนำ หรือแนวทางให้เลือกอย่างน้อย 3

คำแนะนำ หรือ 3 แนวทาง พร้อมข้อดี ข้อเสีย เพื่อให้ผู้ใช้พิจารณา

ตัวอย่าง Prompt ปรับปรุง CLAUDE.md ให้เหมาะสม (ถ้ามี)

ตัวอย่าง

ปัจจุบัน ควรปรับปรุง CLAUDE.md อะไรเพิ่มเติมบ้าง

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill สำหรับสร้าง Technical Stack (optional)

สร้าง agent และ agent skill

สำหรับสร้างเอกสาร tech stack รวมถึงช่วยเลือก tech stack ที่เหมาะสม

โดยการถามผู้ใช้งาน แบบเข้มข้น เพื่อให้ทราบรายละเอียดทั้งหมด

เพื่อสามารถแนะนำ tech stack ที่เหมาะสมได้

หากมีส่วนไหนไม่ชัดเจน ให้ถามผู้ใช้งานเพิ่มเติม พร้อมกับมีคำแนะนำ หรือแนวทางให้เลือกอย่างน้อย 3

คำแนะนำ หรือ 3 แนวทาง พร้อมข้อดี ข้อเสีย เพื่อให้ผู้ใช้พิจารณา

ตัวอย่าง Prompt ปรับปรุง Sub Agent / Agent Skill สำหรับ Architecture, API Spec & DB Spec, Detailed Detail หลังเลือก Technical Stack แล้ว (optional)

ตัวอย่าง

ปรับปรุง agent และ agent skill

ในส่วนของ architecture, api spec, db spec, และ detailed design

ให้เหมาะสม และลงรายละเอียดยิ่งขึ้น จากข้อมูล technical stack ที่มี

Mini Workshop:

สร้าง Non-Functional Requirement

ใช้ Prompt เพื่อสร้าง และหาข้อมูลเกี่ยวกับ Non-Functional Requirement ของระบบ

ตัวอย่าง Prompt เพื่อตรวจสอบ และสร้างเอกสารเรื่อง Non-Functional Requirement

1

จาก requirement, backlog, feature-list, test case, test report, high-level design, detailed design

ควรมี non-functional requirement อะไรบ้าง แนะนำ พร้อมเหตุผล

ถ้ามีหลายทางเลือก ให้แนะนำทางเลือกอื่นๆ พร้อมข้อดี ข้อเสีย

แสดงผลเป็นตาราง เพื่อให้อ่านง่าย พร้อมมีสรุปภาพรวม

ถ้าไม่แน่ใจ หรือต้องการข้อมูลเพิ่มเติม ให้ถาม ห้ามเดา

โดยยังไม่ต้องสร้างเอกสาร แสดงข้อมูลเพื่อให้รีวิว และตัดสินใจก่อน



2

ปรับปรุง หรือสร้างเอกสารที่เกี่ยวข้องจากข้อมูล Non-Functional Requirement

Mini Workshop:

ทดลอง Explore เกี่ยวกับ Technology ที่เหมาะสม

ใช้ Prompt เพื่อหาข้อมูลเกี่ยว Technology ที่เหมาะสมกับระบบ

ตัวอย่าง Prompt เพื่อสอบถามเรื่อง Technology ที่เหมาะสม

จาก requirement, backlog, feature-list, test case, test report, high-level design, detailed design
ของโครงการในปัจจุบัน

แนะนำเทคโนโลยีที่เหมาะสมมาอย่างน้อย 5 ตัวเลือก สำหรับใช้ในการพัฒนา Frontend และ Backend
โดยทำเป็นตารางเปรียบเทียบ ด้วยเทคนิค Trade-off Matrix หรือ Weighted Scoring Model
พร้อมสรุปผล และข้อดี ข้อเสีย แต่ละตัวเลือก และแนะนำเทคโนโลยีใด

โดยยังไม่ต้องสร้างเอกสาร แสดงข้อมูลเพื่อให้รีวิว และตัดสินใจก่อน

ต้องการข้อมูลให้ถามเพิ่ม ห้ามเดา

ซึ่งมีข้อจำกัดอื่นๆ ดังต่อไปนี้

1. ...
2. ...
3. ...

Mini Workshop:

สร้าง Project Plan

สร้าง Sub Agent และ Agent Skill สำหรับแบ่ง Phase ในการทำงาน
รวมถึงใช้ Prompt เพื่อสร้าง Project Plan

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill สำหรับสร้าง Project Planner

สร้าง agent และ agent skill

สำหรับสร้าง หรือปรับปรุง เอกสารเพื่อแบ่ง plan / phase / release

และการแต่งงานย่อย ต่อ phase (task list)

จาก requirement, backlog, feature-list, และข้อมูลอื่นๆ ที่เกี่ยวข้อง ทั้งหมด หรือระบุ ได้

พร้อมทั้งมีคำแนะนำ หรือแนวทางให้เลือกอย่างน้อย 3 คำแนะนำ หรือ 3 แนวทาง

พร้อมข้อดี ข้อเสีย เพื่อให้ผู้ใช้พิจารณา

หากมีส่วนไหนไม่ชัดเจน หรือต้องการข้อมูล ห้ามเดา ให้ถามผู้ใช้งานเพิ่มเติม

ตัวอย่าง Prompt

เพื่อสอบถามและวางแผนในระยะยาว

1

จาก requirement, backlog, feature-list, test case, test report, high-level design, detailed design ของโครงการในปัจจุบัน

ช่วยวางแผนแบ่ง Phase สำหรับการพัฒนา ว่าควรมีอะไรก่อนหลัง MVP Phase, Next Phase, Future Phase หรืออื่นๆ ที่เกี่ยวข้อง เป็นตารางเพื่อให้ง่ายต่อการอ่าน แต่ควรมีสรุบบื้องต้นของเหตุผลในการแบ่ง Phase

โดยยังไม่ต้องสร้างเอกสาร แสดงข้อมูลเพื่อให้รีวิว และตัดสินใจก่อน

ถ้ามีงานชิ้นใดใหญ่เกิน สามารถแบ่ง Phase ได้

ต้องการข้อมูลให้ถามเพิ่ม ห้ามเดา



2

ปรับปรุง หรือสร้างเอกสารที่เกี่ยวข้องจากข้อมูล แผนการพัฒนา

ตัวอย่าง Prompt

เพื่อสอบถามและวางแผนเพิ่มเติม

หลังมีการเพิ่ม Requirement Spec รวมถึงข้อมูลอื่นๆ ที่เกี่ยวข้อง

1

จาก requirement, backlog, feature-list, test case, test report, high-level design, detailed design
ใหม่ที่สร้างขึ้น และยังไม่มีในแผนงาน

ช่วยวางแผนเพิ่มเติม ว่าควรอยู่ในแผนการพัฒนาส่วนใด

หรืออื่นๆ ที่เกี่ยวข้อง เป็นตารางเพื่อให้ง่ายต่อการอ่าน แต่ควรมีสรุปรเบื่องต้นของเหตุผลในการแบ่ง Phase

โดยยังไม่ต้องสร้างเอกสาร แสดงข้อมูลเพื่อให้รีวิว และตัดสินใจก่อน

ถ้ามีงานชิ้นใดใหญ่เกิน สามารถแบ่ง Phase ได้

ต้องการข้อมูลให้ถามเพิ่ม ห้ามเดา



2

ปรับปรุง หรือสร้างเอกสารที่เกี่ยวข้องจากข้อมูล แผนการพัฒนา

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill Orchestrator สำหรับสั่งทำงานครั้งเดียวเกี่ยวกับ Requirement

สร้าง sub agent และ agent skill สำหรับเป็น orchestrator

เพื่อทำงานต่อเนื่องกันโดยไม่ต้องเรียกทีละขั้นตอน

ในส่วนของงาน requirement -> backlog -> feature-list -> user-journey -> acceptance-criteria +
test-case + test plan

ตัวอย่าง Prompt สร้าง Sub Agent / Agent Skill Orchestrator สำหรับสั่งทำงานครั้งเดียวเกี่ยวกับ Tech Spec

สร้าง sub agent และ agent skill สำหรับเป็น orchestrator

เพื่อทำงานต่อเนื่องกันโดยไม่ต้องเรียกทีละขั้นตอน

ในส่วนของการงาน architecture -> api spec + database schema -> detailed design -> review non-functional requirement

Q&A